Project Definition Guide

Creating Workspace Projects in Customer Journey Analytics

Last Updated: March 2026

This guide provides comprehensive information on creating and updating Workspace Project definitions in Adobe Customer Journey Analytics (CJA) using the MCP Server tools.

- 1 Creating Workspace Projects in Customer Journey Analytics
- 2
- 3 Introduction
  - 3.1 Source Schema
- 4 projectBody Wrapper (Required for API)
- 5 Top-Level Definition Structure
  - 5.1 Minimal Valid Skeleton
  - 5.2 Full Skeleton with Common Optional Fields
- 6 The Entity Pattern
  - 6.1 Valid Entity Types
  - 6.2 Entity with Metadata
- 7 Date Range Definitions
  - 7.1 Where Date Ranges Are Used
  - 7.2 Two Forms of DateRange Entities
  - 7.3 Definition Formula Syntax
    - 7.3.1 Rolling Date Anchors
    - 7.3.2 Time Units
    - 7.3.3 Arithmetic
    - 7.3.4 Fixed Dates
    - 7.3.5 Day-of-Week Filter
  - 7.4 Common Date Range Formulas
  - 7.5 How to Build a Formula
  - 7.6 Calendar Types
- 8 Object Hierarchy
  - 8.1 Workspace
  - 8.2 Panel
  - 8.3 Panel Layout (Side-by-Side Panels)
  - 8.4 Panel segmentGroups (Drop-Down Filters)
    - 8.4.1 selectedComponentOptions (Pre-Selected Filter Values)
    - 8.4.2 runAs: segment vs breakdown
  - 8.5 Panel-Level Configs (Auto-Generating Panels)
    - 8.5.1 NextOrPreviousItemPanelConfig
    - 8.5.2 quickInsightsPanelConfig
  - 8.6 GenericSubPanel
    - 8.6.1 SubPanel Position Properties
    - 8.6.2 SubPanel Layout (Grid Layouts Within a Panel)
    - 8.6.3 SubPanel Descriptions
    - 8.6.4 Quill Delta Structure
- 9 FreeformReportlet (Most Common)
  - 9.1 Structure
  - 9.2 columnTree (FreeformColumnNode)
  - 9.3 Sort
  - 9.4 Pagination
  - 9.5 FreeformTableSettings
  - 9.6 dimensionSettings (FreeformTableDimensionSettings)
    - 9.6.1 Multiple Dimensions (Flat/Hierarchical Table)
  - 9.7 FreeformReportlet — Required vs Optional Fields
  - 9.8 Breakdowns (Recursive)
  - 9.9 staticRows (Pre-Populated Rows)
    - 9.9.1 staticRow Object
    - 9.9.2 Example: Segments as Static Rows
    - 9.9.3 Example: Metrics / CalculatedMetrics as Static Rows (Metrics-as-Rows Pattern)
    - 9.9.4 Example: DateRanges as Static Rows (Time Comparison Pattern)
  - 9.10 Complete Freeform Table Example
- 10 Visualization Type Reference
  - 10.1 Type Reference Table
  - 10.2 Common Chart Properties
  - 10.3 BarReportlet Full Properties
  - 10.4 Fallout Visualization
    - 10.4.1 Structure
    - 10.4.2 Checkpoint Object
    - 10.4.3 Complete Fallout Example
  - 10.5 Flow Visualization
    - 10.5.1 Structure
    - 10.5.2 FlowConfig Object (Required)

        - 10.5.3 rootPrefixColumn — Nested FlowColumn Chain (Critical)
        - 10.5.4 FlowColumn Object
        - 10.5.5 DimensionItem IDs for expandedNodes
        - 10.5.6 Complete Flow Example
    - 10.6 ComboReportlet (Combo Chart with Time Comparison)
        - 10.6.1 Structure
        - 10.6.2 lineComparisons — Time Comparison Object
        - 10.6.3 DateRange definition Syntax
        - 10.6.4 Data Table for Combo Chart
    - 10.7 Cohort Table Visualization
        - 10.7.1 Structure
        - 10.7.2 CohortCriteria Object
        - 10.7.3 MetricComparison Object
        - 10.7.4 CohortTable Object
        - 10.7.5 Complete Cohort Table Example
    - 10.8 Histogram Visualization
        - 10.8.1 Structure
        - 10.8.2 Linked Table: Empty and Hidden
        - 10.8.3 Minimal Histogram Reportlet
    - 10.9 Journey Canvas Visualization
        - 10.9.1 Structure
        - 10.9.2 mfeJson Object
        - 10.9.3 Node Object
        - 10.9.4 Edge Object
        - 10.9.5 Filters Object
        - 10.9.6 Metric References in mfeJson
        - 10.9.7 Complete Journey Canvas Example
    - 10.10 Key Metric Summary Visualization
        - 10.10.1 Structure
        - 10.10.2 Data Table for Key Metric Summary (Critical)
        - 10.10.3 DateRange definition Syntax
    - 10.11 Map Visualization
        - 10.11.1 Structure
        - 10.11.2 Complete Map Example
    - 10.12 Venn Visualization
        - 10.12.1 Structure
        - 10.12.2 Venn-Specific Properties
        - 10.12.3 Linked FreeformReportlet
        - 10.12.4 Key Notes
    - 10.13 Summary Change Visualization
        - 10.13.1 Structure
        - 10.13.2 Complete Example
    - 10.14 Section Header Visualization
        - 10.14.1 Structure
        - 10.14.2 Complete Example
    - 10.15 TextReportlet
    - 10.16 Area and Stacked Chart Variants
        - 10.16.1 AreaReportlet
        - 10.16.2 AreaStackedReportlet
        - 10.16.3 BarStackedReportlet
        - 10.16.4 Example (BarStackedReportlet)
    - 10.17 Scatter Plot Visualization
        - 10.17.1 Structure
        - 10.17.2 Complete Example
    - 10.18 Bullet Chart Visualization
        - 10.18.1 Structure
        - 10.18.2 Complete Example
- 11 Linking Visualizations to Data (linkedSourceId)
    - 11.1 How It Works
    - 11.2 SubPanel Array Order = Visual Order
    - 11.3 visualizationIndex Convention
    - 11.4 Example: Freeform Table + Line Chart
    - 11.5 Example: Freeform Table + Summary Number
    - 11.6 lockedSelection & FreeformBreakdownSelection (Critical)
        - 11.6.1 Structure
        - 11.6.2 Selecting a Top-Level Row
        - 11.6.3 Selecting a Breakdown Row (Single Cell)
- 12 Common Patterns
    - 12.1 1. Simple Table (One Metric, One Dimension)
    - 12.2 2. Table with Multiple Metrics
    - 12.3 3. Table + Line Chart (Linked)
    - 12.4 4. Table with Date Range Comparison Columns
    - 12.5 5. Bar Chart Linked to a Table
    - 12.6 6. Flat Table with Multiple Dimensions + Bar Chart
    - 12.7 7. Panel with Dynamic Dropdown Filter
    - 12.8 8. Fallout Visualization
    - 12.9 9. Flow Visualization
    - 12.10 10. Combo Chart with Time Comparison
    - 12.11 11. Cohort Table

- 12.12 12. Histogram
- 12.13 13. Journey Canvas
- 12.14 14. Key Metric Summary
- 12.15 15. Map
- 12.16 16. Next or Previous Item Panel
- 12.17 17. Quick Insights Panel
- 12.18 18. Venn Diagram
- 12.19 19. Dashboard Grid Layout (Multi-Row, Multi-Column)

- 13 Best Practices
  - 13.1 1. Always Include Required Fields
  - 13.2 2. Use UUIDs for All IDs
  - 13.3 3. Match linkedSourceId Correctly
  - 13.4 4. Use breakdownByPosition Intentionally
  - 13.5 5. Validate Before Saving
  - 13.6 6. Keep columnWidths Aligned
- 14 Validation
  - 14.1 Recommended: npx ajv-cli (supports $ref)
  - 14.2 Online Validator
  - 14.3 curl (Assertible API — limited)
  - 14.4 Python (local)
- 15 Common Errors and Troubleshooting
  - 15.1 1. Missing __entity__: true
  - 15.2 2. Wrong Entity Type
  - 15.3 3. linkedSourceId Mismatch
  - 15.4 4. Invalid Sort columnId
  - 15.5 5. Schema Validation Fails on additionalProperties
  - 15.6 6. Bare Definition Without projectBody Wrapper
  - 15.7 7. Quill Delta Escaping
  - 15.8 8. lockedSelection: "None of the provided types can parse this json"
  - 15.9 9. Multi-Dimension Table Fails or Shows Wrong Layout
  - 15.10 10. Flow: "Cannot read properties of undefined (reading 'numColumns')"
  - 15.11 11. Summary Number Selecting Multiple Cells
- 16 Template-Specific Features (autorun & autoCompareTimePeriod)
  - 16.1 autorun (Panel Config Property)
  - 16.2 autoCompareTimePeriod (Column Node Property — Templates Only)
  - 16.3 autorunTemplateFunctions Flow
- 17 API Reference
  - 17.1 Useful Expansions
- 18 Future Tasks
  - 18.1 1. Histogram — Autorun Support (Resolved: Not Supported)
  - 18.2 2. Flow — Autorun Support (Resolved: Not Supported)
  - 18.3 3. Attribution — Full Documentation
  - 18.4 4. Experimentation, A4T, and Segment Comparison Panels

Introduction

A Workspace Project definition is a JSON structure that describes the full layout and content of a CJA Analysis Workspace project. It defines which panels, tables, charts, metrics, dimensions, and segments appear in the project, and how they are configured.

Source Schema

The authoritative JSON Schema lives at:

```
https://awaascicdprodva7.blob.core.windows.net/deploy/schemas/project/projectSchema_v96.json
```

> ***Important***: *Download with `curl --compressed` — the CDN returns compressed content.*

---

projectBody Wrapper (Required for API)

The project definition JSON is **not** used directly. It must be nested inside a `projectBody` wrapper object when passed to `upsertProject`. The API will reject a bare definition.

```
{
  "definition": { ... },
  "description": "",
  "rsid": "dv_YOUR_DATA_VIEW_ID",
  "type": "project"
}
```

Property	Type	Required	Description
`definition`	object	Yes	The project definition JSON (described below)
`rsid`	string	Yes	Data view ID (must match `reportSuite.id` in panels)
`type`	string	Yes	Always `"project"`
`description`	string	No	Project description text
`name`	string	No	Project display name

Top-Level Definition Structure

The project definition is the `definition` field inside the `projectBody`. The top-level properties are:

Property	Type	Required	Description
`version`	string	Yes	Schema version. Current value:`"96"`
`workspaces`	array of Workspace	Yes	The workspace(s) in the project
`colorScheme`	ColorScheme	No	Color palette for visualizations
`countRepeatInstances`	boolean	No	Whether to count repeat instances
`showAnnotations`	boolean	No	Show annotations globally
`viewDensity`	string	No	`"expanded"` or `"compact"`
`currentWorkspaceIndex`	number	No	Index of the active workspace
`isCurated`	boolean	No	Whether the project is curated
`showTableOfContents`	boolean	No	Show a table of contents sidebar listing all panels by name. Useful for long projects with many panels.

Minimal Valid Skeleton

```
{
  "definition": {
    "version": "96",
    "workspaces": [
      {
        "id": "WORKSPACE-UUID",
        "name": "",
        "panels": []
      }
    ]
  },
  "description": "",
  "rsid": "dv_YOUR_DATA_VIEW_ID",
  "type": "project"
}
```

Full Skeleton with Common Optional Fields

```
{
  "definition": {
    "additionalCuratedComponents": [],
    "countRepeatInstances": true,
    "currentWorkspaceIndex": 0,
    "customColorSchemes": [],
    "isCurated": false,
    "showAnnotations": true,
    "version": "96",
    "viewDensity": "expanded",
    "workspaces": []
  },
  "description": "",
  "rsid": "dv_YOUR_DATA_VIEW_ID",
  "type": "project"
}
```

The Entity Pattern

This is the single most important pattern in the schema. Every CJA component (metric, dimension, segment, date range) is referenced through an Entity object:

```
{
  "__entity__": true,
  "id": "metrics/visits",
  "type": "Metric"
}
```

Valid Entity Types

type Value	id Format	Example
"Metric"	metrics/{metric_id}	"metrics/visits", "metrics/occurrences"
"CalculatedMetric"	cm{orgId}_{uuid}	"cm088D2D305B47DD820A495E75@AdobeOrg_5f84a2a0..."
"Dimension"	variables/{dimension_id}	"variables/daterangeday", "variables/page"
"DimensionItem"	variables/{dim_id}::{item}	"variables/daterangeday::1260201"
"DateRange"	Preset name or UUID	"thisMonth", "lastFullMonth", "last7days", "6507a5b3...". For custom/rolling ranges, use __metaData__.definition with a formula (see Date Range Definitions section).
"Segment"	Segment UUID	"s300012345_6478..."
"ReportSuite"	Data view ID	"dv_636c0814a52b..."
"AttributionTemplate"	Attribution ID	"allocation-instance"

> ***CalculatedMetric entities work identically to Metric entities*** *everywhere in the project definition — column tree nodes, reportlet components, cohort criteria, histogram components, etc. Use `"type": "CalculatedMetric"` instead of `"Metric"` and the calculated metric's full ID. No other changes are needed.*

Entity with Metadata

Entities can include an optional `__metaData__` object for display names:

```
{
  "id": "variables/daterangeday",
  "__entity__": true,
  "type": "Dimension",
  "__metaData__": {
    "name": "Day"
  }
}
```

Date Range Definitions

Date ranges appear throughout project JSON — on panels, in column tree nodes, in reportlet configs (Combo, Key Metric Summary), and as panel-level `dateRange` entities. Understanding how to construct them is essential.

Where Date Ranges Are Used

Location	Purpose	Supports Presets	Supports Formulas
panel.dateRange	The panel's overall date range	Yes (`"id": "thisMonth"`)	Yes (`"definition": "tm-1m/tm"`)
config.dateRange	Panel config date range (e.g., Quick Insights)	Yes	Yes

`columnTree` node with `type: "DateRange"`	Date comparison columns in FreeformReportlet	Yes (saved UUID)	Yes
`reportlet.primaryDateRange` / `comparisonDateRange`	Key Metric Summary, Combo Chart comparisons	No	Yes (formula only)

Two Forms of DateRange Entities

1. Preset (named) date range — uses a known preset `id`:

```
{
  "id": "thisMonth",
  "__entity__": true,
  "type": "DateRange",
  "__metaData__": { "name": "This month" }
}
```

2. Custom/rolling date range — uses `__metaData__.definition` with a formula string:

```
{
  "__entity__": true,
  "type": "DateRange",
  "__metaData__": {
    "definition": "td-6d/td+1d"
  }
}
```

Note: Custom date ranges typically omit the `id` field (or use a generated UUID). The `definition` formula is what CJA actually evaluates.

Definition Formula Syntax

The formula format is: **`start/end`**

With an optional filter: **`start/end|filter`**

Formulas are evaluated relative to the report execution date (when the report was supposed to run, not when it actually executes if queued).

Rolling Date Anchors

Anchor	Meaning
`td`	Start of**this day**
`tw`	Start of**this week**
`tm`	Start of**this month**
`tq`	Start of**this quarter**
`ty`	Start of**this year**
`th`	Start of**this hour**
`ti`	Start of**this minute**

Rule: If there is a `t` in the formula, it must always come first. Read `t` as "start of THIS...".

Time Units

Unit	Meaning
`s`	Second
`i`	Minute
`h`	Hour
`d`	Day
`w`	Week
`m`	Month

q	Quarter
y	Year

Arithmetic

- `td-1d` = today minus 1 day = yesterday's start
- `tm+1m` = this month's start plus 1 month = next month's start
- `ty-1y` = this year's start minus 1 year = last year's start

Month arithmetic edge cases:

- Sep 26 - 1m = Aug 26
- Oct 31 + 1m = Nov 30 (clamped to month end)
- Oct 31 + 1m + 1m = Dec 30 (not Dec 31 — each step clamps independently)
- Oct 31 + 2m = Dec 31 (single step: goes straight to 2 months later)

Fixed Dates

Use ISO 8601 format: `YYYY-MM-DDTHH:MM:SS`

```
2015-01-06T00:00:00/2015-03-27T23:59:59
```

You can mix fixed and rolling: `2015-01-06T00:00:00/td` (Jan 6, 2015 to today).

Day-of-Week Filter

Append `|{dow:[...]}` to filter by day of week:

Value	Day
1	Sunday
2	Monday
3	Tuesday
4	Wednesday
5	Thursday
6	Friday
7	Saturday

Example: `ty-1y/ty|{dow:[1,7]}` = last full year, weekends only.

Common Date Range Formulas

Name	Formula	Description
Today	`td/td+1d`	Start of today to start of tomorrow
Yesterday	`td-1d/td`	Start of yesterday to start of today
This hour	`th/th+1h`	
This minute	`ti/ti+1i`	
Last 7 days	`td-6d/td+1d`	6 days ago through today
Last 14 days	`td-13d/td+1d`	
Last 30 days	`td-29d/td+1d`	
Last 60 days	`td-59d/td+1d`	
Last 90 days	`td-89d/td+1d`	
This week	`tw/tw+1w`	
Last full week	`tw-1w/tw`	

Last 2 full weeks	`tw-2w/tw`	
Last 3 full weeks	`tw-3w/tw`	
Last 4 full weeks	`tw-4w/tw`	
Last 53 full weeks	`tw-53w/tw`	
This week last year	`tw-52w/tw-51w`	Same week number, previous year
This month	`tm/tm+1m`	
Last full month	`tm-1m/tm`	Previous month
Last 2 full months	`tm-2m/tm`	
Last 3 full months	`tm-3m/tm`	
Last 6 full months	`tm-6m/tm`	
Last 12 full months	`tm-12m/tm`	
Last 13 full months	`tm-13m/tm`	
This year	`ty/ty+1y`	
Last full year	`ty-1y/ty`	
Last full year weekends	`ty-1y/ty|{dow:[1,7]}`	
Last full year weekdays	`ty-1y/ty|{dow:[2,3,4,5,6]}`	
This day last month	`td-1m/td-1m+1d`	Same day number, previous month
This day last year	`td-1y/td-1y+1d`	Same day number, previous year

How to Build a Formula

1. **Identify the period type**: day, week, month, quarter, year
2. **Choose the anchor**: `td`, `tw`, `tm`, `tq`, `ty`
3. **Calculate start**: Subtract the desired lookback from the anchor (e.g., `tm-3m` for 3 months ago)
4. **Calculate end**: Usually the anchor itself (for "last N full periods") or anchor + 1 unit (for "this period" or "last N days including today")
5. **Pattern for "last N full [units]"**: `t[unit] - N[unit] / t[unit]`
6. **Pattern for "last N [days] including today"**: `td - (N-1)d / td + 1d`
7. **Pattern for "this [period]"**: `t[unit] / t[unit] + 1[unit]`

Calendar Types

CJA supports multiple calendar systems that affect how week/month/quarter boundaries are calculated:

Calendar	Description
Gregorian	Standard calendar. Months and weeks follow normal rules.
Modified Gregorian	User selects the starting month and day of week for the year. The selected day becomes the first day of the week. Months are still standard months.
NRF (National Retail Federation)	4-5-4 week pattern per quarter (4 weeks, 5 weeks, 4 weeks). Sunday-to-Saturday weeks. Months are defined by number of weeks, not calendar dates. Uses an**anchor date** (standard: Feb 4) that must fall in the first week of the year. The anchor date determines the starting weekday. A **"Big Year"** occurs when an extra week is needed to keep the anchor date in the first week of the following year.
QRS	Same as NRF but follows a 4-4-5 pattern instead of 4-5-4.
Custom NRF/QRS	Same as NRF/QRS but with a user-selected anchor date instead of the standard Feb 4.

The calendar type affects how `tw`, `tm`, `tq`, `ty` boundaries are resolved. For Gregorian calendars the boundaries are straightforward; for NRF/QRS the "month" and "week" boundaries follow the retail pattern.

Object Hierarchy

Projects follow a strict nesting hierarchy:

```
ProjectDefinition
   workspaces[]
         Workspace
               panels[]
                     Panel
                           dateRange (Entity)
                           reportSuite (Entity)
                           segmentGroups[]
                           subPanels[]
                                 GenericSubPanel
                                       reportlet (one visualization type)
```

Workspace

```
{
  "id": "UUID",
  "name": "",
  "panels": []
}
```

Property	Type	Description
`id`	string	Unique UUID for this workspace
`name`	string	Display name (can be empty)
`panels`	array of Panel	Panels in the workspace

Panel

```
{
  "collapsed": false,
  "dateRange": { "__entity__": true, "id": "thisMonth", "type": "DateRange" },
  "datesAreRelativeToPanel": false,
  "description": "",
  "id": "PANEL-UUID",
  "name": "Freeform",
  "position": { "autoSize": true, "width": 100, "x": 0, "y": 0 },
  "reportSuite": { "__entity__": true, "id": "dv_636c0814a52b0b1044ce89a3", "type": "ReportSuite" },
  "segmentGroups": [],
  "subPanels": [],
  "type": "panel"
}
```

Property	Type	Description
`id`	string	Unique UUID
`name`	string	Panel display name
`description`	string	Panel description text
`dateRange`	Entity	Date range for this panel
`reportSuite`	Entity	Data view / report suite
`segmentGroups`	array	Segment filters applied to the panel
`subPanels`	array of GenericSubPanel	Visualizations in this panel
`type`	string	Always `"panel"` for standard panels
`collapsed`	boolean	Whether the panel is collapsed
`position`	GridPosition	Layout position (see Panel Layout below)

`config`	PanelConfig	Optional. Panel-level config for special panel types (e.g., Next or Previous Item, Quick Insights). May have empty or pre-built `subPanels` depending on the config type.

Panel Layout (Side-by-Side Panels)

Panels use a `position` object with `width` (percentage of workspace width) and `x` (horizontal offset as percentage) to control layout. By default, panels are full-width (`width: 100, x: 0`) and stack vertically.

To place panels **side by side**, set `width: 50` on each and offset the right panel with `x: 50`:

```
{
  "panels": [
    {
      "id": "PANEL-LEFT",
      "name": "This Month",
      "dateRange": { "id": "thisMonth", "__entity__": true, "type": "DateRange" },
      "position": { "autoSize": true, "width": 50, "x": 0, "y": 0 },
      "subPanels": [ ... ]
    },
    {
      "id": "PANEL-RIGHT",
      "name": "Last Month",
      "dateRange": { "__entity__": true, "type": "DateRange", "__metaData__": { "definition": "tm-1m/tm" } },
      "position": { "autoSize": true, "width": 50, "x": 50, "y": 0 },
      "subPanels": [ ... ]
    }
  ]
}
```

Property	Type	Description
`width`	number	Panel width as percentage of workspace (default `100`). Use `50` for two side-by-side panels.
`x`	number	Horizontal offset as percentage. Left panel:`0`, right panel: `50`.
`y`	number	Vertical offset. CJA auto-calculates, but panels on the same row should share the same `y`.
`autoSize`	boolean	`true` = auto-calculate height based on content
`autoHeight`	number	Calculated height in pixels. Can be omitted when `autoSize: true` — CJA will compute it.

Key notes:

- Panels on the same row should have the same `y` value (e.g., both `y: 0`)
- Each side-by-side panel operates independently — its own `dateRange`, `segmentGroups`, and `subPanels`
- This is useful for date range comparisons: put "This Month" and "Last Month" side by side with identical visualizations but different panel `dateRange` values
- `panel.dateRange` supports both **preset IDs** (`"id": "thisMonth"`) and **rolling formulas** via `__metaData__.definition` (e.g., `"definition": "tm-1m/tm"` — no `id` needed)
- Multiple rows of side-by-side panels can be created by incrementing `y` values

Panel segmentGroups (Drop-Down Filters)

Panel-level `segmentGroups` create interactive drop-down filters above the panel. Each group can hold static segments (`ComponentGroup`) or a dynamic dimension (`DynamicComponentGroup`):

```
{
  "segmentGroups": [
    {
      "dynamicDimension": {
        "id": "variables/page",
        "__entity__": true,
        "type": "Dimension",
        "__metaData__": { "name": "Page" }
      },
      "groupName": "",
      "hasNoFilter": true,
      "selectedComponentOptions": [],
      "showGroupName": true
    }
  ]
}
```

Property	Type	Description
`dynamicDimension`	Entity	The dimension to populate the dropdown with
`groupName`	string	Display label for the dropdown (empty = use dimension name)
`hasNoFilter`	boolean	`true` = start with no filter (show all data); `false` = require a selection
`selectedComponentOptions`	array	Pre-selected values (empty = none selected)
`showGroupName`	boolean	Whether to display the group name label
`runAs`	string	**`"segment"`** (default) or **`"breakdown"`**. Controls how the dropdown filters the panel — as a segment filter or as a breakdown on the data

selectedComponentOptions (Pre-Selected Filter Values)

To pre-select specific dimension items in a dropdown filter, add entries to `selectedComponentOptions`. Each entry references a `DimensionItem` entity with `isActive: true`:

```
{
  "selectedComponentOptions": [
    {
      "component": {
        "id": "variables/geocountry::US",
        "__entity__": true,
        "type": "DimensionItem",
        "__metaData__": { "name": "US" }
      },
      "isActive": true
    }
  ]
}
```

Field	Type	Description
`component`	Entity	A `DimensionItem` entity from the dropdown's dimension
`isActive`	boolean	`true` = this item is selected on load

When `selectedComponentOptions` is `[]` (empty) and `hasNoFilter: true`, the dropdown starts with no filter applied (all data shown). When pre-selected values are provided, the panel immediately filters to those values on load.

runAs: segment vs breakdown

- **Omit `runAs`** or set `"runAs": "segment"` — the dropdown acts as a **segment filter**, narrowing the data in all visualizations to only rows matching the selected dimension values.
- **`"runAs": "breakdown"`** — the dropdown acts as a **breakdown filter**, breaking down the data by the selected dimension values rather than filtering them out.

Panel-Level Configs (Auto-Generating Panels)

Some panel types use a `config` object on the panel itself (not on a reportlet) to control panel behavior. These fall into two categories:

- **Empty subPanels**: The panel auto-generates its own visualizations (e.g., NextOrPreviousItem). `subPanels` should be `[]`.
- **Pre-built subPanels**: The panel includes starter subpanels that are managed by the config (e.g., Quick Insights). `subPanels` contains the initial visualization scaffolding.

NextOrPreviousItemPanelConfig

Shows what dimension items came before or after a specific item. The panel auto-generates a table of next/previous items.

```
{
  "config": {
    "container": "visitors",
    "dimension": {
      "id": "variables/daterangeday",
      "__entity__": true,
      "type": "Dimension",
      "__metaData__": { "name": "Day" }
    },
    "dimensionItem": {
      "id": "variables/daterangeday::1260201",
      "__entity__": true,
      "type": "DimensionItem",
      "__metaData__": { "name": "Mar 1, 2026" }
    },
    "direction": "next",
    "registeredConfigs": {},
    "type": "NextOrPreviousItemPanelConfig"
  },
  "subPanels": []
}
```

Field	Type	Description
`type`	string	**Required.** `"NextOrPreviousItemPanelConfig"`
`dimension`	Entity	The dimension to analyze (e.g., Day, Page, Identity)
`dimensionItem`	Entity	The specific DimensionItem to start from. Uses `type: "DimensionItem"` with a specific item ID.
`direction`	string	`"next"` = show what came after; `"previous"` = show what came before
`container`	string	Counting container:`"visitors"`, `"visits"`, etc.
`registeredConfigs`	object	Typically empty `{}`

Key notes:

- The `config` sits on the **panel** (not on a subpanel/reportlet)
- `subPanels` must be `[]` — the panel auto-generates its own visualizations
- `dimensionItem` requires a specific DimensionItem entity with a real ID (e.g., `variables/daterangeday::1260201`)
- Panel `autoHeight: 536` is standard
- Panel `name` is typically `"Next or previous item"`

quickInsightsPanelConfig

An exploratory panel that lets users interactively build queries by selecting dimensions, metrics, and segments in a guided builder UI. Unlike NextOrPreviousItem, Quick Insights **includes two pre-built subPanels** (a freeform table + a bar chart).

Important: The `type` is **camelCase** (`"quickInsightsPanelConfig"`), unlike other panel configs that use PascalCase.

```
{
  "config": {
    "dateRange": {
      "id": "thisMonth", "__entity__": true, "type": "DateRange",
      "__metaData__": { "name": "This month" }
    },
    "dimensions": {
      "first": { "show": true },
      "hideFirstItem": false,
      "second": { "show": false },
      "third": { "show": false }
    },
    "metrics": {
      "first": { "show": true },
      "hideFirstItem": false,
      "second": { "show": false },
      "third": { "show": false }
    },
    "registeredConfigs": {},
    "segments": {
      "first": { "logicalOperator": "and", "show": true },
      "hideFirstItem": false,
      "second": { "logicalOperator": "and", "show": false },
      "third": { "logicalOperator": "and", "show": false }
    },
    "type": "quickInsightsPanelConfig"
  }
}
```

Field	Type	Description
`type`	string	**Required.** `"quickInsightsPanelConfig"` (camelCase!)
`dateRange`	Entity	Date range for the Quick Insights analysis
`dimensions`	object	Slot configuration for up to 3 dimension drop zones. Each slot (`first`, `second`, `third`) has `show` (boolean). Top level has `hideFirstItem` (boolean).
`metrics`	object	Slot configuration for up to 3 metric drop zones. Same structure as `dimensions`.
`segments`	object	Slot configuration for up to 3 segment drop zones. Each slot adds `logicalOperator` (`"and"` / `"or"`).
`registeredConfigs`	object	Typically empty `{}`

SubPanels (required — NOT empty):

Quick Insights requires two subpanels with specific IDs and `isQuickInsightsSubPanel: true`:

SubPanel	ID	Type	Position	Description
Freeform table	`"quickinsights-freeform-subpanel"`	`FreeformReportlet`	Left half (`width: 50, x: 0`)	Empty data table; `linkedSourceId: ""`
Visualization	`"quickinsights-viz-subpanel"`	`BarReportlet`	Right half (`width: 50, x: 50`)	Bar chart; `linkedSourceId: "quickinsights-freeform-subpanel"`

Key notes:

- `config.type` is **camelCase** `"quickInsightsPanelConfig"` — using PascalCase will cause a "None of the provided types can parse this json" error
- Both subpanels must have `isQuickInsightsSubPanel: true`
- The subpanel IDs must be exactly `"quickinsights-freeform-subpanel"` and `"quickinsights-viz-subpanel"`
- The freeform table is empty (no column tree nodes, no dimension settings) — CJA populates it when the user interacts
- The viz subpanel uses `autoSize: false` with `fixedHeight: 419`
- Panel `autoHeight: 837` (taller than standard to accommodate side-by-side layout)
- Panel `name` is typically `"Quick insights"`

GenericSubPanel

Each subpanel contains one visualization (reportlet):

```
{
  "collapsed": false,
  "description": "",
  "id": "SUBPANEL-UUID",
  "isQuickInsightsSubPanel": false,
  "linkedSourceId": "",
  "position": { "autoSize": true, "width": 100, "x": 0, "y": 0 },
  "reportlet": { ... },
  "type": "genericSubPanel",
  "visible": true,
  "visualizationIndex": 1
}
```

Property	Type	Description
`id`	string	Unique UUID
`name`	string	Optional display name
`description`	string	Rich text description (Quill delta JSON string — see below)
`linkedSourceId`	string	ID of the data-source subpanel this viz reads from (see Linking Visualizations)
`reportlet`	object	The visualization definition (one of the reportlet types)
`type`	string	Always `"genericSubPanel"`
`visible`	boolean	Whether the subpanel is visible
`visualizationIndex`	number	Ordering index

SubPanel Position Properties

The `position` object controls where a subpanel appears within its panel:

Property	Type	Description
`width`	number	Width as percentage of the panel (e.g.,`100` = full width, `50` = half, `33.33` = third)
`x`	number	Horizontal offset as percentage. Left column:`0`, right column: `50`, etc.
`y`	number	Vertical offset in pixels.**Set `0` for all subpanels** — CJA auto-computes vertical positions based on the flow layout.
`autoSize`	boolean	`true` = auto-calculate height from content. `false` = use `fixedHeight` instead.
`autoHeight`	number	Calculated height in pixels. Can be omitted when `autoSize: true` — CJA computes it.
`fixedHeight`	number	Explicit height in pixels. Only used when `autoSize: false`. Use this to force a visualization to a specific height (e.g., making a chart span multiple rows).

Key notes:

- `width` and `x` work the same way as panel positioning — they control the horizontal grid
- You can safely set `y: 0` for every subpanel. CJA's flow-based layout engine automatically stacks subpanels vertically based on their height and width. You do NOT need to manually calculate `y` offsets.
- `fixedHeight` is the primary tool for controlling visualization height. Combine it with `autoSize: false` to override CJA's default auto-sizing.

SubPanel Layout (Grid Layouts Within a Panel)

Subpanels within a single panel can be arranged in a grid using `width` and `x`, just like side-by-side panels. This is the recommended approach for dashboard-style layouts.

Side-by-side subpanels (two columns):

```
{ "position": { "autoSize": true, "width": 50, "x": 0, "y": 0 } }
{ "position": { "autoSize": true, "width": 50, "x": 50, "y": 0 } }
```

Three-column row (e.g., summary number tiles):

```
{ "position": { "autoSize": true, "width": 33.33, "x": 0, "y": 0 } }
{ "position": { "autoSize": true, "width": 33.34, "x": 33.33, "y": 0 } }
{ "position": { "autoSize": true, "width": 33.33, "x": 66.67, "y": 0 } }
```

Tall visualization spanning multiple rows: Set `autoSize: false` with `fixedHeight` on the left, and stack normal-height subpanels on the right. CJA flows the right-column subpanels vertically while the left visualization holds its height:

```
// Left: tall bar chart spanning ~2 rows
{ "position": { "autoSize": false, "fixedHeight": 650, "width": 50, "x": 0, "y": 0 } }
// Right top: horizontal bar
{ "position": { "autoSize": true, "width": 50, "x": 50, "y": 0 } }
// Right bottom: freeform table (flows below the horizontal bar)
{ "position": { "autoSize": true, "width": 50, "x": 50, "y": 0 } }
```

This produces:

```
                    Horizontal Bar
   Bar Chart
   (tall, 650px)    Freeform Table
```

Typical `fixedHeight` values:

- Summary tiles: `150–200`
- Standard chart: `300–400`
- Tall / row-spanning chart: `600–800`

SubPanel Descriptions

The `description` field is the correct way to add annotation text below a visualization title. The format is a serialized Quill delta JSON string:

```
{
  "description": "{\"ops\":[{\"insert\":\"Column descriptions: \\n\"},{\"attributes\":{\"bold\":true},\"insert\":
\"No Conflict\"},{\"insert\":\": When a journey has a single entrance.\\n\"},{\"attributes\":{\"bold\":true},\"
insert\":\"Higher Priority\"},{\"insert\":\": When a journey started because it was more important than any other
feature.\\n\"}]}"
}
```

This renders as rich text below the visualization title:

> ***Column descriptions:***
> ***No Conflict****: When a journey has a single entrance.*
> ***Higher Priority****: When a journey started because it was more important than any other feature.*

Quill Delta Structure

The delta `ops` array supports:

Attribute	Effect
`{"insert": "text"}`	Plain text
`{"attributes": {"bold": true}, "insert": "text"}`	**Bold text**
`{"attributes": {"italic": true}, "insert": "text"}`	*Italic text*
`{"insert": "\\n"}`	Line break

The same Quill delta format is used by `TextReportlet.textContent` (see Visualization Type Reference).

FreeformReportlet (Most Common)

The FreeformReportlet is a data table — the most common visualization. Most other chart types read their data from a FreeformReportlet via `linkedSourceId`.

Structure

```
{
  "type": "FreeformReportlet",
  "columnTree": { ... },
  "freeformTable": { ... },
  "hideTitle": false,
  "showAnnotations": true,
  "showControls": true,
  "isConfigVisible": true,
  "isReadOnly": false
}
```

columnTree (FreeformColumnNode)

The `columnTree` defines the metric columns. Each metric is a node in `nodes[]`:

```
{
  "columnTree": {
    "_computedValues": [],
    "id": "CT-ROOT",
    "name": "",
    "nodes": [
      {
        "_computedValues": [],
        "component": {
          "id": "metrics/occurrences",
          "__entity__": true,
          "type": "Metric",
          "__metaData__": { "name": "Events" }
        },
        "id": "COL-001",
        "name": "Events",
        "nodes": [],
        "visible": true
      }
    ],
    "visible": true
  }
}
```

> ***Required fields***: *The root `columnTree` node and each child node* ***must*** *include `id`, `_computedValues`, and `visible` to pass schema validation. Without these, `additionalProperties: false` on `FreeformColumnNode` will reject the object.*

Nested `nodes` within a column node represent date-range comparison columns or secondary breakdowns on that metric.

freeformTable

The `freeformTable` defines the row dimension, breakdowns, sorting, and pagination:

```json
{
  "freeformTable": {
    "alignDatesForTimeDimension": true,
    "breakdowns": [],
    "collapsed": false,
    "columnWidths": [100, 100],
    "dimensionSettings": [
      {
        "dimension": {
          "id": "variables/daterangeday",
          "__entity__": true,
          "type": "Dimension",
          "__metaData__": { "name": "Day" }
        },
        "id": "DIM-001"
      }
    ],
    "pagination": {
      "currentPage": 0,
      "viewBy": 10
    },
    "settings": {
      "breakdownByPosition": false,
      "rowBasedPercentages": false,
      "totalsType": "allVisits"
    },
    "sort": {
      "asc": true,
      "columnId": "",
      "labelColumn": true
    },
    "staticRows": [],
    "statistics": {
      "functions": [],
      "ignoreZeros": true
    }
  }
}
```

Sort

Property	Type	Description
`asc`	boolean	`true` = ascending, `false` = descending
`columnId`	string	ID of the column node to sort by; empty string = default
`labelColumn`	boolean	`true` = sort by dimension labels (alphabetical)

Pagination

Property	Type	Description
`currentPage`	number	Zero-indexed page number
`viewBy`	number	Rows per page (e.g., 5, 10, 50, 400)

FreeformTableSettings

Property	Type	Description
`breakdownByPosition`	boolean	`true` = breakdowns follow whatever rows are currently displayed (recommended default). `false` = breakdowns are pinned to a specific row item identified by `parentItemId`.

`rowBasedPercentages`	boolean	`true` = show percentages relative to row total
`totalsType`	string	`"allVisits"` or `"columnSum"`
`showThumbnails`	boolean	Show thumbnail previews
`staticIncludeNone`	boolean	Include "None" row in static rows

dimensionSettings (FreeformTableDimensionSettings)

The `dimensionSettings` array defines the row dimensions for the table. Each entry is a `FreeformTableDimensionSettings` object:

Property	Type	Description
`dimension`	Entity	The dimension Entity (with `id`, `type`, etc.)
`id`	string	Unique identifier for this dimension setting
`includeNone`	boolean	Whether to include "None" rows
`search`	Search	Optional search/filter on dimension items

> ***Schema note***: *The schema uses `dimensionSettings` (not `dimensions`). Each entry wraps the dimension Entity inside a `{ "dimension": Entity, "id": "..." }` object. This is a common source of validation errors.*

Multiple Dimensions (Flat/Hierarchical Table)

Adding multiple entries to `dimensionSettings` creates a flat table where the second dimension auto-breaks down under the first. This adds extra label columns — one per dimension — so `columnWidths` must account for all columns (dimension labels + metric columns).

Example: Day + Hour with one metric = 3 columns (Day label, Hour label, Events):

```
"dimensionSettings": [
  {
    "dimension": { "id": "variables/daterangeday", "__entity__": true, "type": "Dimension" },
    "id": "DIM-001"
  },
  {
    "dimension": { "id": "variables/daterangehour", "__entity__": true, "type": "Dimension" },
    "id": "DIM-002"
  }
],
"columnWidths": [33.33, 33.33, 33.33]
```

Key rules for multi-dimension tables:

- `columnWidths` must have `N dimensions + N metrics` entries (e.g., 2 dimensions + 1 metric = 3 entries)
- Use `advancedSortRules` with the primary dimension's ID and `"componentType": "dimension"` for proper ordering:

```
"sort": {
  "advancedSortRules": [
    { "asc": true, "columnId": "DIM-001", "componentType": "dimension" }
  ],
  "asc": true,
  "columnId": "",
  "labelColumn": true
}
```

- You can set `position.y: 0` for all subpanels — CJA auto-computes vertical positions. Manual y offsets are not required (see SubPanel Position Properties).

FreeformReportlet — Required vs Optional Fields

Many FreeformReportlet fields have safe defaults and can be omitted for simpler definitions. Use this reference to decide what to include.

Always required:

Field	Notes

`type`	`"FreeformReportlet"`
`columnTree`	Must include `id`, `_computedValues: []`, `visible: true`, and `nodes[]`
`columnTree.nodes[].id`	Unique per column
`columnTree.nodes[].component`	Entity (typically Metric or CalculatedMetric; can also be Segment or DateRange when using metrics-as-rows pattern).**Rule:** metrics must all be in columns OR all in rows — never both.
`columnTree.nodes[].nodes`	`[]` for leaf columns
`columnTree.nodes[]._computedValues`	`[]`
`columnTree.nodes[].visible`	`true`
`freeformTable.dimensionSettings`	At least one dimension. Each needs `dimension` (Entity) and `id`. **Exception:** can be `[]` when using the metrics-as-rows `staticRows` pattern (see staticRows).
`freeformTable.pagination`	`{ "currentPage": 0, "viewBy": N }`
`freeformTable.sort`	At minimum `{ "asc": true, "columnId": "", "labelColumn": true }`
`freeformTable.staticRows`	`[]`
`freeformTable.statistics`	`{ "functions": [], "ignoreZeros": true }`

Optional (safe to omit):

Field	Default Behavior
`freeformTable.breakdowns`	`[]` — no breakdowns
`freeformTable.collapsed`	`false`
`freeformTable.columnWidths`	Auto-calculated. Provide `[N, N, ...]` only for explicit sizing.
`freeformTable.parentItemIds`	`[]`
`freeformTable.selectionCoordinates`	`[]`
`freeformTable.settings`	Defaults to `breakdownByPosition: false`, `rowBasedPercentages: false`, `totalsType: "allVisits"`
`freeformTable.alignDatesForTimeDimension`	`true` when using time dimensions
`columnTree.nodes[].name`	Display name — defaults to the component name
`columnTree.nodes[].dataSettings`	Only needed for KMS/Combo backing tables
`columnTree.nodes[].selectionCoordinates`	Only needed for KMS/Combo backing tables
`columnTree.nodes[].tableCellDisplay`	Only needed for KMS/Combo backing tables
`hideTitle`	`false`
`isConfigVisible`	`true`
`showAnnotations`	`true`
`showControls`	`true`

> ***Exception:*** *KMS and Combo Chart backing tables require the full verbose column nodes with `dataSettings`, `selectionCoordinates`, and `tableCellDisplay` on every node. See those sections for details.*

Breakdowns (Recursive)

The `breakdowns` array contains nested `FreeformTable` objects with the same structure. Each breakdown targets a specific parent row:

```
{
  "breakdowns": [
    {
      "dimensionSettings": [
        {
          "dimension": {
            "id": "variables/mobiledevicenumber",
            "__entity__": true,
            "type": "Dimension",
            "__metaData__": { "name": "Mobile Device Number" }
          },
          "id": "BD-DIM-001"
        }
      ],
      "parentItemIds": ["2250661301"],
      "breakdowns": [],
      "pagination": { "currentPage": 0, "viewBy": 5 },
      "sort": { "asc": false, "columnId": "COL-001", "labelColumn": false },
      "settings": { "breakdownByPosition": false, "rowBasedPercentages": false, "totalsType": "allVisits" },
      "staticRows": [],
      "statistics": { "functions": [], "ignoreZeros": true }
    }
  ]
}
```

staticRows (Pre-Populated Rows)

By default, `staticRows` is `[]` and the freeform table auto-populates rows from the dimension. To **pin specific rows** to the table, add entries to `staticRows`. Each entry becomes a fixed row that always appears regardless of date range or filter changes. Static rows can use any of these entity types: `Segment`, `DimensionItem`, `Metric`, `CalculatedMetric`, or `DateRange`.

> ***Metric Placement Rule:*** *A table must always have at least one metric, but metrics must live exclusively in the* ***columns*** *or exclusively in the* ***rows*** *— never both. If metrics are in the columns (the typical case), the rows can be dimensions, segments, date ranges, or dimension items. If metrics are in the rows (metrics-as-rows pattern), the columns should be segments or date ranges — not other metrics.*

staticRow Object

Field	Type	Description
`component`	Entity	The entity to display as a row. Can be `Segment`, `DimensionItem`, `Metric`, `CalculatedMetric`, or `DateRange`.
`id`	string	Unique identifier for this static row
`dataSettings`	object	Optional. Advanced item limit and search. Standard default below.

Example: Segments as Static Rows

```
{
  "staticRows": [
    {
      "component": {
        "id": "s_YOUR_SEGMENT_ID_1",
        "__entity__": true,
        "type": "Segment",
        "__metaData__": { "name": "Web Analytics Data" }
      },
      "dataSettings": {
        "advancedItemLimit": 5,
        "advancedItemSearch": { "alwaysExcludedItems": [], "operator": "AND", "rules": [] }
      },
      "id": "SR-001"
    },
    {
      "component": {
        "id": "s_YOUR_SEGMENT_ID_2",
        "__entity__": true,
        "type": "Segment",
        "__metaData__": { "name": "Call Center Data" }
      },
      "dataSettings": {
        "advancedItemLimit": 5,
        "advancedItemSearch": { "alwaysExcludedItems": [], "operator": "AND", "rules": [] }
      },
      "id": "SR-002"
    }
  ]
}
```

Example: Metrics / CalculatedMetrics as Static Rows (Metrics-as-Rows Pattern)

When you want to display **metrics as rows** instead of as columns, use `Metric` or `CalculatedMetric` entities in `staticRows` and leave `dimensionSettings` **empty** (`[]`). The column header typically uses a `Segment` (e.g., "All Data") to provide the aggregation context:

```
{
  "columnTree": {
    "_computedValues": [],
    "id": "CT-1",
    "name": "",
    "visible": true,
    "nodes": [
      {
        "_computedValues": [],
        "component": {
          "id": "All_Visits",
          "__entity__": true,
          "type": "Segment",
          "__metaData__": { "name": "All Data" }
        },
        "id": "COL-001",
        "name": "All Data",
        "nodes": [],
        "visible": true
      }
    ]
  },
  "freeformTable": {
    "dimensionSettings": [],
    "staticRows": [
      {
        "component": {
          "id": "cm_YOUR_ORG_ID_calcmetric1",
          "__entity__": true,
          "type": "CalculatedMetric",
          "__metaData__": { "name": "Avg Proc Time (sec)" }
        },
        "dataSettings": {
          "advancedItemLimit": 5,
          "advancedItemSearch": { "alwaysExcludedItems": [], "operator": "AND", "rules": [] }
        },
        "id": "SR-001"
      },
      {
        "component": {
          "id": "cm_YOUR_ORG_ID_calcmetric2",
          "__entity__": true,
          "type": "CalculatedMetric",
          "__metaData__": { "name": "Errors" }
        },
        "dataSettings": {
          "advancedItemLimit": 5,
          "advancedItemSearch": { "alwaysExcludedItems": [], "operator": "AND", "rules": [] }
        },
        "id": "SR-002"
      },
      {
        "component": {
          "id": "metrics/occurrences",
          "__entity__": true,
          "type": "Metric",
          "__metaData__": { "name": "Reports Run" }
        },
        "dataSettings": {
          "advancedItemLimit": 5,
          "advancedItemSearch": { "alwaysExcludedItems": [], "operator": "AND", "rules": [] }
        },
        "id": "SR-003"
      }
    ]
  }
}
```

Example: DateRanges as Static Rows (Time Comparison Pattern)

When you want to compare a metric across different time periods, use `DateRange` entities in `staticRows` with a metric column. Like the metrics-as-rows pattern, set `dimensionSettings: []`:

```
{
  "columnTree": {
    "_computedValues": [],
    "id": "CT-1",
    "name": "",
    "visible": true,
    "nodes": [
      {
        "_computedValues": [],
        "component": {
          "id": "cm_YOUR_ORG_calcmetric_errors",
          "__entity__": true,
          "type": "CalculatedMetric",
          "__metaData__": { "name": "Errors" }
        },
        "id": "COL-001",
        "name": "Errors",
        "nodes": [],
        "visible": true
      }
    ]
  },
  "freeformTable": {
    "dimensionSettings": [],
    "staticRows": [
      {
        "component": {
          "id": "lastFullWeek",
          "__entity__": true,
          "type": "DateRange",
          "__metaData__": { "name": "Last Week" }
        },
        "dataSettings": {
          "advancedItemLimit": 5,
          "advancedItemSearch": { "alwaysExcludedItems": [], "operator": "AND", "rules": [] }
        },
        "id": "SR-001"
      },
      {
        "component": {
          "id": "lastFullMonth",
          "__entity__": true,
          "type": "DateRange",
          "__metaData__": { "name": "Last month" }
        },
        "dataSettings": {
          "advancedItemLimit": 5,
          "advancedItemSearch": { "alwaysExcludedItems": [], "operator": "AND", "rules": [] }
        },
        "id": "SR-002"
      },
      {
        "component": {
          "id": "today",
          "__entity__": true,
          "type": "DateRange",
          "__metaData__": { "name": "Today" }
        },
        "dataSettings": {
          "advancedItemLimit": 5,
          "advancedItemSearch": { "alwaysExcludedItems": [], "operator": "AND", "rules": [] }
        },
        "id": "SR-003"
      }
    ]
  }
}
```

Key notes:

- When `staticRows` is non-empty, the table shows **only** these rows (the dimension auto-population is replaced)
- Each `component` follows the standard Entity pattern — `Segment`, `DimensionItem`, `Metric`, `CalculatedMetric`, or `DateRange`
- `dataSettings` is optional but recommended for consistency
- `id` must be unique within the table
- Static rows with `Segment` entities show the metric values filtered to each segment
- Static rows with `DimensionItem` entities show specific dimension values (e.g., specific page names, specific dates)
- Static rows with `Metric` or `CalculatedMetric` entities flip the table: metrics become row labels and the column(s) provide the aggregation context (typically a Segment or DateRange). When using this pattern, set `dimensionSettings: []` (empty)
- Static rows with `DateRange` entities compare a metric across different time windows (e.g., Last Week vs Last Month vs Today). The column(s) must be metrics. Set `dimensionSettings: []` (empty)

- **Remember the metric placement rule:** metrics must all be in the columns OR all in the rows — never both

Complete Freeform Table Example

A freeform table showing "Events" by Day:

```
{
  "type": "genericSubPanel",
  "id": "DATA-SUBPANEL-UUID",
  "linkedSourceId": "",
  "reportlet": {
    "type": "FreeformReportlet",
    "columnTree": {
      "id": "CT-ROOT",
      "name": "",
      "_computedValues": [],
      "visible": true,
      "nodes": [
        {
          "id": "COL-001",
          "_computedValues": [],
          "visible": true,
          "component": {
            "id": "metrics/occurrences",
            "__entity__": true,
            "type": "Metric",
            "__metaData__": { "name": "Events" }
          },
          "name": "Events",
          "nodes": []
        }
      ]
    },
    "freeformTable": {
      "dimensionSettings": [
        {
          "dimension": {
            "id": "variables/daterangeday",
            "__entity__": true,
            "type": "Dimension",
            "__metaData__": { "name": "Day" }
          },
          "id": "DIM-001"
        }
      ],
      "breakdowns": [],
      "pagination": { "currentPage": 0, "viewBy": 400 },
      "sort": { "asc": true, "columnId": "", "labelColumn": true },
      "settings": {
        "breakdownByPosition": false,
        "rowBasedPercentages": false,
        "totalsType": "allVisits"
      },
      "staticRows": [],
      "statistics": { "functions": [], "ignoreZeros": true }
    }
  }
}
```

Visualization Type Reference

Every reportlet has a `type` string that determines the visualization. All visualization types share these common fields:

- `hideTitle` (boolean) — hide the visualization title
- `showAnnotations` (boolean) — show annotation markers
- `showControls` (boolean) — show the control bar
- `isConfigVisible` (boolean) — show the configuration panel
- `intelligentCaptions` (object) — AI-generated caption settings

Type Reference Table

Type String	Category	Key Unique Properties
`FreeformReportlet`	Table	`columnTree`, `freeformTable`, `advancedMode`, `advancedSettings`

`LineReportlet`	Trend	`trendlineModel`, `dualYAxis`, `allowAnomaliesToRescale`, `allowForecastingToRescale`, `showAnomalies`, `showForecast`, `showTrendline`, `movingAverageWindow`, `forceZero`, `anchorToLeft`
`BarReportlet`	Bar	`normalized`, `numItems`, `dualYAxis`
`BarStackedReportlet`	Bar	`normalized`, `numItems`
`HorizontalBarReportlet`	Bar	`normalized`, `numItems`
`HorizontalBarStackedReportlet`	Bar	`normalized`
`AreaReportlet`	Trend	`dualYAxis`, `showAnomalies`
`AreaStackedReportlet`	Trend	`normalized`
`ComboReportlet`	Trend	`barMetric`, `dimension`, `lineComparisons` (with `comparisonType`, `comparisonValue`), `config` (ComboChart Config), `dualYAxis`, `forceZero` (see Combo section below)
`DonutReportlet`	Part-to-whole	`numItems`, `showLegend`
`ScatterReportlet`	Relationship	`numItems`
`HistogramReportlet`	Distribution	`component` (metric Entity), `name`, `hasLimit`, `numItems`, `usePercentage`. Linked to an **empty** hidden FreeformReportlet. Optional: `bucketCount`, `bucketSize`, `startingBucket`, `countingMethod` (see Histogram section below)
`BulletReportlet`	KPI	—
`KeyMetricSummaryReportlet`	KPI	`keyMetric`, `primaryDateRange`, `comparisonDateRange` (all on reportlet + `config`), `showComparison`, `showTrendlines`, `summaryDisplayType`, `displayType`. Linked table needs two-column date range structure. (see Key Metric Summary section below)
`SummaryNumberReportlet`	KPI	`abbreviateValue`, `abbreviatePrecision`, `summarizeValue`, `summarizeValueBy`, `usePercentage`, `lockType`, `lockedSelection` (see dedicated section)
`SummaryChangeReportlet`	KPI	`abbreviateValue`, `usePercentage`
`TextReportlet`	Text	`textContent` (rich text Quill delta JSON string)
`MapReportlet`	Geo	`metricComponent` (metric Entity), `mapType` (`"bubbles"` / `"heatmap"`), `mapStyle`, `colorTheme`, `isLatLng`, `center`, `zoom`, `bearing`, `pitch`, `clusterRadius`. Self-contained, no `linkedSourceId`. (see Map section below)
`TreemapReportlet`	Part-to-whole	—
`VennReportlet`	Overlap	`metrics` (array of Metric entities), `segments` (array of Segment entities — each becomes a circle), `contextForSegmentConversion` (`"visits"`, `"visitors"`). Linked to an **empty** hidden FreeformReportlet. (see Venn section below)
`CohortReportlet`	Retention	`cohortTable`, `inclusionCriteria`, `returnCriteria` (CohortCriteria with `metricComparisons`), `granularityValue`, `container`, `churn`, `dimensionColumnEnabled`, `rollingCohortEnabled` (see Cohort section below)
`Fallout`	Pathing	`checkpoints`, `falloutContainer`, `includeAllVisitsCheckpoint`, `segments` (see Fallout section below)
`Flow`	Pathing	`config` (FlowConfig with `entryComponent`, `numColumns`, `numNodes`), `rootComponent`, `rootPrefixColumn` (nested FlowColumn chain), `flowContainer`, `disableTruncation` (see Flow section below)
`JourneyCanvasReportlet`	Pathing	`mfeJson` (graph with `nodes`, `edges`, `filters`, `container`, `primaryMetric`, `secondaryMetric`, `isSequential`). Self-contained, no `linkedSourceId`. (see Journey Canvas section below)
`GuidedAnalysisReportlet`	Guided	`mfeJson` (micro-frontend config)
`SectionHeaderReportlet`	Layout	—

Common Chart Properties

Most chart reportlets (Line, Bar, Area, etc.) share:

Property	Type	Description
`lockType`	string	`"positions"` or `"items"` — how the chart selects rows
`numItems`	number	Number of items to display (default: 14)
`legendVisible`	boolean	Show the chart legend
`legendMappings`	object	Custom legend-to-series mappings
`disabledLegends`	array	Legend items to hide
`normalized`	boolean	Normalize to 100% (stacked charts)
`dualYAxis`	boolean	Enable a second Y-axis
`usePercentage`	boolean	Show values as percentages
`useRowBasedPercentages`	boolean	Use row-based percentage calculation

BarReportlet Full Properties

When creating a `BarReportlet`, include all these properties for it to render correctly. A minimal set may fail silently or render incorrectly:

```
{
  "type": "BarReportlet",
  "disabledLegends": [],
  "dualYAxis": false,
  "hasLimit": false,
  "hideTitle": false,
  "intelligentCaptions": {
    "captions": [],
    "hasLoaded": false,
    "hiddenCaptions": [],
    "isExpanded": false
  },
  "isConfigVisible": true,
  "legendMappings": { "entries": [] },
  "legendVisible": true,
  "lockType": "positions",
  "normalized": false,
  "numItems": 14,
  "showAnnotations": true,
  "showControls": true,
  "showVizSelectorOnSubPanel": false,
  "usePercentage": false,
  "useRowBasedPercentages": false
}
```

Fallout Visualization

A `Fallout` reportlet shows a funnel of sequential checkpoints and the drop-off between each step. Unlike chart types (Line, Bar, etc.), Fallout is a standalone visualization — it does **not** use `linkedSourceId` and holds its own data configuration.

Structure

Property	Type	Description
`type`	string	`"Fallout"`
`checkpoints`	array	Array of `Checkpoint` objects defining each step in the funnel
`falloutContainer`	string	Counting container:`"visitors"`, `"visits"`, `"opportunities"`, `"buyinggroups"`, `"accounts"`, `"globalaccounts"`
`includeAllVisitsCheckpoint`	boolean	Whether to include the "All Visits" starting point
`segments`	array	Array of Segment entities for comparison (typically includes `All_Visits`)
`isConfigVisible`	boolean	Show configuration panel
`legendMappings`	object	`{ "entries": [] }`
`hideTitle`	boolean	Hide the visualization title

`showAnnotations`	boolean	Show annotations
`showControls`	boolean	Show controls

Checkpoint Object

Each checkpoint represents a step in the funnel:

```
{
  "items": [
    {
      "id": "metrics/occurrences",
      "__entity__": true,
      "type": "Metric",
      "__metaData__": { "name": "Events" }
    }
  ],
  "name": "",
  "within": "eventual"
}
```

Field	Type	Description
`items`	array	Array of Entity objects — the component(s) for this checkpoint. Can be Metrics, Segments, or Dimension items.
`name`	string	Optional display name for the checkpoint
`within`	string	`"eventual"` = steps don't need to be consecutive. Other values constrain timing.

Complete Fallout Example

A 3-step funnel tracking people who had Events Events Events:

```
{
  "id": "SP-FALLOUT",
  "linkedSourceId": "",
  "position": { "autoHeight": 295, "autoSize": true, "width": 100, "x": 0, "y": 0 },
  "reportlet": {
    "checkpoints": [
      {
        "items": [
          { "id": "metrics/occurrences", "__entity__": true, "type": "Metric", "__metaData__": { "name": "Events"
} }
        ],
        "name": "",
        "within": "eventual"
      },
      {
        "items": [
          { "id": "metrics/occurrences", "__entity__": true, "type": "Metric", "__metaData__": { "name": "Events"
} }
        ],
        "name": "",
        "within": "eventual"
      },
      {
        "items": [
          { "id": "metrics/occurrences", "__entity__": true, "type": "Metric", "__metaData__": { "name": "Events"
} }
        ],
        "name": "",
        "within": "eventual"
      }
    ],
    "falloutContainer": "visitors",
    "hideTitle": false,
    "includeAllVisitsCheckpoint": true,
    "intelligentCaptions": { "captions": [], "hasLoaded": false, "hiddenCaptions": [], "isExpanded": false },
    "isConfigVisible": true,
    "legendMappings": { "entries": [] },
    "segments": [
      { "id": "All_Visits", "__entity__": true, "type": "Segment", "__metaData__": { "name": "All Visits" } }
    ],
    "showAnnotations": true,
    "showControls": true,
    "type": "Fallout"
  },
  "swatchColor": "#ED6332",
  "type": "genericSubPanel",
  "visible": true,
  "visualizationIndex": 0
}
```

Key notes:

- `linkedSourceId` must be `""` (empty) — Fallout is self-contained
- Checkpoint `items` can use Metrics, Segments, or Dimension item entities
- `falloutContainer: "visitors"` tracks people through the funnel; `"visits"` tracks sessions
- `segments` with `All_Visits` provides the comparison baseline

Flow Visualization

A `Flow` reportlet shows pathing analysis — what happened before and/or after a specific dimension. Like Fallout, Flow is standalone and does **not** use `linkedSourceId`.

Structure

Property	Type	Description
`type`	string	`"Flow"`
`config`	FlowConfig	**Required.** Configuration object with entry/exit component, column/node counts, and container. Must include `"type": "reportletConfig"`.
`rootComponent`	Entity	The center dimension of the flow (e.g., Day)
`rootPrefixColumn`	FlowColumn	Nested column chain defining the flow path. Depth must match `config.numColumns`.

`rootSuffixColumn`	FlowColumn	(Optional) Columns after the root. Omit for entry flows.
`flowContainer`	string	`"visitors"`, `"visits"`, etc.
`disableTruncation`	boolean	`true` recommended — prevents node label truncation
`includeRepeatInstances`	boolean	Whether to count repeat visits to the same node
`numNodes`	number	Number of nodes per column (e.g., 5)
`hideTitle`	boolean	Hide the visualization title
`isConfigVisible`	boolean	Show the configuration panel
`showAnnotations`	boolean	Show annotations
`showControls`	boolean	Show controls
`showGrayPopovers`	boolean	Show popover tooltips on nodes

FlowConfig Object (Required)

The `config` object tells the flow how to behave. Without it, the flow fails with errors about missing `numColumns`.

```
{
  "config": {
    "disableTruncation": true,
    "entryComponent": {
      "id": "variables/daterangeday",
      "__entity__": true,
      "type": "Dimension",
      "__metaData__": { "name": "Day" }
    },
    "flowContainer": "visitors",
    "includeRepeatInstances": false,
    "limitEntryExit": true,
    "numColumns": 3,
    "numNodes": 5,
    "type": "reportletConfig"
  }
}
```

Field	Type	Description
`entryComponent`	Entity	The dimension to start the flow from (entry flow). Use `exitComponent` instead for exit flows.
`exitComponent`	Entity	The dimension the flow ends at (exit flow). Mutually exclusive with `entryComponent`.
`flowContainer`	string	Counting container:`"visitors"`, `"visits"`, etc.
`includeRepeatInstances`	boolean	Count repeat instances
`limitEntryExit`	boolean	`true` = limit to first entry/last exit only
`numColumns`	number	**Required.** Number of columns in the flow (typically 3)
`numNodes`	number	Number of nodes per column
`type`	string	**Required.** Must be `"reportletConfig"`

rootPrefixColumn — Nested FlowColumn Chain (Critical)

The `rootPrefixColumn` defines the flow's column structure as a nested chain of `FlowColumn` objects via `childColumn`. The nesting depth must match `config.numColumns`. Each level has `dimension`, `expandedNodes`, `limit`, and `search`.

The `expandedNodes` pattern is critical for the flow to render all columns:

- **Level 1 (outermost)**: `expandedNodes` contains a single entry with the **Dimension** entity (type `"Dimension"`). This triggers expansion from column 1 column 2.
- **Level 2 (middle)**: `expandedNodes` contains **DimensionItem** entities (type `"DimensionItem"`) with specific item IDs. This triggers expansion from column 2 column 3. You must provide real dimension item IDs — a Dimension entity does not work here.
- **Level 3 (innermost)**: `expandedNodes` is `[]` — leaf level, no further expansion.

Without proper `expandedNodes` entries, the flow renders only one column (the root node) and does not expand.

FlowColumn Object

```
{
  "dimension": {
    "id": "variables/daterangeday",
    "__entity__": true,
    "type": "Dimension",
    "__metaData__": { "name": "Day" }
  },
  "expandedNodes": [],
  "limit": 5,
  "search": {
    "alwaysExcludedItems": [],
    "operator": "AND",
    "rules": []
  }
}
```

Field	Type	Description
`dimension`	Entity	The dimension for this column
`expandedNodes`	array	Array of FlowNode objects (`{ "component": Entity }`) that are expanded. See pattern above.
`limit`	number	Max nodes to show in this column
`search`	Search	Filter rules for the column
`childColumn`	FlowColumn	Next nested column (recursive)

DimensionItem IDs for expandedNodes

Level 2 `expandedNodes` require **DimensionItem** entities with specific IDs. For date dimensions, the ID format is `variables/daterangeday::<number>` where the number encodes the date. These IDs are data-view specific. To obtain them:

1. Create the flow with empty `expandedNodes` at level 2 (it will show 2 columns instead of 3)
2. Open in CJA, let it render and interact with it
3. Export/describe the project to capture the populated DimensionItem IDs
4. Use those IDs in future project definitions

Alternatively, use known IDs from a working project with the same data view and date range.

Complete Flow Example

An entry flow starting from Day, with 3 columns and 5 nodes per column:

```
{
  "id": "SP-FLOW",
  "linkedSourceId": "",
  "position": { "autoHeight": 657, "autoSize": true, "width": 100, "x": 0, "y": 0 },
  "reportlet": {
    "config": {
      "disableTruncation": true,
      "entryComponent": {
        "id": "variables/daterangeday",
        "__entity__": true,
        "type": "Dimension",
        "__metaData__": { "name": "Day" }
      },
      "flowContainer": "visitors",
      "includeRepeatInstances": false,
      "limitEntryExit": true,
      "numColumns": 3,
      "numNodes": 5,
      "type": "reportletConfig"
    },
    "disableTruncation": true,
    "flowContainer": "visitors",
    "hideTitle": false,
    "includeRepeatInstances": false,
    "intelligentCaptions": { "captions": [], "hasLoaded": false, "hiddenCaptions": [], "isExpanded": false },
    "isConfigVisible": true,
    "numNodes": 5,
    "rootComponent": {
      "id": "variables/daterangeday",
      "__entity__": true,
```

```
    "type": "Dimension",
    "__metaData__": { "name": "Day" }
  },
  "rootPrefixColumn": {
    "childColumn": {
      "childColumn": {
        "dimension": {
          "id": "variables/daterangeday",
          "__entity__": true,
          "type": "Dimension",
          "__metaData__": { "name": "Day" }
        },
        "expandedNodes": [],
        "limit": 5,
        "search": { "alwaysExcludedItems": [], "operator": "AND", "rules": [] }
      },
      "dimension": {
        "id": "variables/daterangeday",
        "__entity__": true,
        "type": "Dimension",
        "__metaData__": { "name": "Day" }
      },
      "expandedNodes": [
        {
          "component": {
            "id": "variables/daterangeday::1260201",
            "__entity__": true,
            "type": "DimensionItem",
            "__metaData__": { "name": "Mar 1, 2026" }
          }
        },
        {
          "component": {
            "id": "variables/daterangeday::1260202",
            "__entity__": true,
            "type": "DimensionItem",
            "__metaData__": { "name": "Mar 2, 2026" }
          }
        },
        {
          "component": {
            "id": "variables/daterangeday::1260203",
            "__entity__": true,
            "type": "DimensionItem",
            "__metaData__": { "name": "Mar 3, 2026" }
          }
        },
        {
          "component": {
            "id": "variables/daterangeday::1260204",
            "__entity__": true,
            "type": "DimensionItem",
            "__metaData__": { "name": "Mar 4, 2026" }
          }
        },
        {
          "component": {
            "id": "variables/daterangeday::1260205",
            "__entity__": true,
            "type": "DimensionItem",
            "__metaData__": { "name": "Mar 5, 2026" }
          }
        }
      ],
      "limit": 5,
      "search": { "alwaysExcludedItems": [], "operator": "AND", "rules": [] }
    },
    "dimension": {
      "id": "variables/daterangeday",
      "__entity__": true,
      "type": "Dimension",
      "__metaData__": { "name": "Day" }
    },
    "expandedNodes": [
      {
        "component": {
          "id": "variables/daterangeday",
          "__entity__": true,
          "type": "Dimension",
          "__metaData__": { "name": "Day" }
        }
      }
    ],
    "limit": 5,
```

```
      "search": { "alwaysExcludedItems": [], "operator": "AND", "rules": [] }
    },
    "showAnnotations": true,
    "showControls": true,
    "showGrayPopovers": true,
    "type": "Flow"
  },
  "swatchColor": "#00C0C7",
  "type": "genericSubPanel",
  "visible": true,
  "visualizationIndex": 1
}
```

Key notes:

- `linkedSourceId` must be `""` (empty) — Flow is self-contained
- `config` is **required** — without it, the flow errors on missing `numColumns`
- `config.type` must be `"reportletConfig"`
- For entry flows, use `config.entryComponent`. For exit flows, use `config.exitComponent`.
- `rootPrefixColumn` nesting depth must match `config.numColumns` (3 levels for 3 columns)
- Level 1 `expandedNodes` uses a **Dimension** entity; Level 2 uses **DimensionItem** entities with real IDs
- Panel `autoHeight: 831`, subpanel `autoHeight: 657` for proper sizing
- `visualizationIndex: 1` (not 0)

ComboReportlet (Combo Chart with Time Comparison)

A `ComboReportlet` shows a bar + line combination chart, commonly used to compare a metric across two time periods (e.g., this month vs last month). Unlike simple chart types, the ComboReportlet has its own `barMetric`, `dimension`, and `lineComparisons` properties — it is **not** just a passthrough from the linked table.

Structure

Property	Type	Description
`type`	string	`"ComboReportlet"`
`barMetric`	Entity	**Required.** The metric shown as bars (e.g., Events). Must be set on both the reportlet and inside `config`.
`dimension`	Entity	**Required.** The dimension for the X-axis (e.g., Day). Must be set on both the reportlet and inside `config`.
`lineComparisons`	array	**Required.** Array of line comparison objects. Must be set on both the reportlet and inside `config`.
`config`	ComboChart Config	**Required.** Configuration object mirroring `barMetric`, `dimension`, `lineComparisons`, and `barMetricFilters`. Must include `"type": "reportletConfig"`.
`barMetricFilters`	SegmentGroup	Filters for the bar metric. Set on both the reportlet and inside `config`.
`dualYAxis`	boolean	Enable a second Y-axis. Typically `false` for time comparison.
`forceZero`	boolean	Force Y-axis to start at zero
`lockType`	string	`"positions"` or `"items"`
`numItems`	number	Number of items to display (default: 14)
`showBarbells`	boolean	Show barbell connectors between comparison points
`showXAxis` / `showYAxis`	boolean	Show/hide axes

lineComparisons — Time Comparison Object

Each entry in `lineComparisons` defines a comparison line. For time-period comparison, use `comparisonType: "time-comparison"` with a `comparisonValue` DateRange entity:

```
{
  "comparisonFilters": {
    "first": { "logicalOperator": "and", "show": true },
    "hideFirstItem": true,
    "second": { "logicalOperator": "and", "show": false },
    "third": { "logicalOperator": "and", "show": false }
  },
  "comparisonType": "time-comparison",
  "comparisonValue": {
    "__entity__": true,
    "type": "DateRange",
    "__metaData__": {
      "name": "Previous month",
      "definition": "tm-1m/tm"
    }
  },
  "id": "LINE-CMP-UUID"
}
```

Field	Type	Description
`comparisonType`	string	`"time-comparison"` for date period comparisons
`comparisonValue`	Entity	DateRange entity**without an `id` field** — uses `__metaData__.definition` instead (e.g., `"tm-1m/tm"` for previous month)
`comparisonFilters`	SegmentGroup	Filter configuration.`hideFirstItem: true` for time comparisons.
`id`	string	Unique identifier for this comparison

DateRange definition Syntax

The `comparisonValue` DateRange uses a `definition` string instead of an `id`. Common values:

Definition	Meaning
`tm-1m/tm`	Previous month (relative to panel date range)
`tm-1w/tm`	Previous week
`tm-1y/tm`	Previous year

Data Table for Combo Chart

The ComboReportlet reads from a linked FreeformReportlet. The table's columnTree must have **two separate metric column nodes** (not one metric with a nested DateRange):

1. **Column 1**: The bar metric (e.g., Events) — `nodes: []`
2. **Column 2**: The same metric again — with a nested DateRange child for the comparison period

```
"nodes": [
  {
    "id": "COL-001",
    "component": { "id": "metrics/occurrences", "__entity__": true, "type": "Metric" },
    "name": "Events",
    "nodes": [],
    "_computedValues": [], "visible": true
  },
  {
    "id": "COL-002",
    "component": { "id": "metrics/occurrences", "__entity__": true, "type": "Metric" },
    "name": "Events",
    "nodes": [
      {
        "id": "COL-003",
        "component": {
          "__entity__": true,
          "type": "DateRange",
          "__metaData__": { "name": "Previous month", "definition": "tm-1m/tm" }
        },
        "name": "Previous month",
        "nodes": [],
        "_computedValues": [], "visible": true
      }
    ],
    "_computedValues": [], "visible": true
  }
]
```

Key differences from simple date comparison (Pattern #4):

- Two **separate** metric column nodes, not one metric with a nested DateRange
- The DateRange entity uses `__metaData__.definition` (e.g., `"tm-1m/tm"`) instead of an `id` like `"lastFullMonth"`
- `columnWidths` needs 3 entries: `[33.33, 33.33, 33.33]` (dimension label + 2 metric columns)
- The data table subpanel should have `visible: false` — the combo chart renders the data visually

Cohort Table Visualization

A `CohortReportlet` shows a retention (or churn) table — tracking whether people who performed an inclusion action return to perform a return action over subsequent time periods. Like Fallout and Flow, the Cohort Table is standalone and does **not** use `linkedSourceId`.

Structure

Property	Type	Description
`type`	string	`"CohortReportlet"`
`cohortTable`	CohortTable	Table layout configuration with `columnWidths` and `selectionCoordinates`
`inclusionCriteria`	CohortCriteria	**Required.** Defines the inclusion metric condition (what action qualifies a person for the cohort)
`returnCriteria`	CohortCriteria	**Required.** Defines the return metric condition (what action counts as a "return")
`granularityValue`	string	**Required.** Time granularity: `"Day"`, `"Week"`, `"Month"`, `"Quarter"`, `"Year"`
`container`	string	Counting container:`"visitors"`, `"visits"`, `"opportunities"`, `"buyinggroups"`, `"accounts"`, `"globalaccounts"`
`churn`	boolean	`false` = retention mode (default), `true` = churn mode (shows who did NOT return)
`dimension`	Entity	Optional custom dimension for the cohort (omit for standard time-based cohort)
`dimensionColumnEnabled`	boolean	Show a dimension column in the table
`dimensionSearch`	Search	Filter rules for the dimension column
`isConfigVisible`	boolean	Show the configuration panel in the UI
`latencyTableEnabled`	boolean	Show a latency table alongside the cohort
`onlyShowPercent`	boolean	Show only percentages (hide raw counts)
`rollingCohortEnabled`	boolean	Use rolling cohort calculation

`roundPercentToNearestWhole`	boolean	Round percentages to whole numbers
`showAnnotations`	boolean	Show annotation markers
`showAveragePercentRow`	boolean	Show the average percent row at the bottom
`showControls`	boolean	Show the control bar

CohortCriteria Object

Both `inclusionCriteria` and `returnCriteria` use the same `CohortCriteria` structure:

```
{
  "metricComparisons": [
    {
      "comparisonType": "ge",
      "metric": {
        "id": "metrics/occurrences",
        "__entity__": true,
        "type": "Metric",
        "__metaData__": { "name": "Events" }
      },
      "value": 1
    }
  ],
  "metricOperator": "and",
  "segments": []
}
```

Field	Type	Description
`metricComparisons`	array	Array of `MetricComparison` objects defining the conditions
`metricOperator`	string	`"and"` or `"or"` — how multiple comparisons are combined
`segments`	array	Optional segment entities to further filter the criteria

MetricComparison Object

Field	Type	Description
`comparisonType`	string	`"ge"` (>=), `"gt"` (>), `"eq"` (=), `"le"` (<=), `"lt"` (<)
`metric`	Entity	The metric entity to compare
`value`	number	The threshold value

CohortTable Object

```
{
  "cohortTable": {
    "columnWidths": [11.11, 11.11, 11.11, 11.11, 11.11, 11.11, 11.11, 11.11, 11.11],
    "selectionCoordinates": []
  }
}
```

Field	Type	Description
`columnWidths`	array	Column width percentages. 9 equal entries (`11.11` each) is the standard default.
`selectionCoordinates`	array	Pre-selected cells (typically empty `[]`)

Complete Cohort Table Example

A retention cohort tracking people with Events >= 1 as both inclusion and return, with Day granularity:

```
{
  "id": "SP-COHORT",
  "linkedSourceId": "",
  "position": { "autoHeight": 616, "autoSize": true, "width": 100, "x": 0, "y": 0 },
  "reportlet": {
    "churn": false,
    "cohortTable": {
      "columnWidths": [11.11, 11.11, 11.11, 11.11, 11.11, 11.11, 11.11, 11.11, 11.11],
      "selectionCoordinates": []
    },
    "container": "visitors",
    "dimensionColumnEnabled": false,
    "dimensionSearch": { "operator": "AND", "rules": [] },
    "granularityValue": "Day",
    "inclusionCriteria": {
      "metricComparisons": [
        {
          "comparisonType": "ge",
          "metric": {
            "id": "metrics/occurrences", "__entity__": true, "type": "Metric",
            "__metaData__": { "name": "Events" }
          },
          "value": 1
        }
      ],
      "metricOperator": "and",
      "segments": []
    },
    "intelligentCaptions": { "captions": [], "hasLoaded": false, "hiddenCaptions": [], "isExpanded": false },
    "isConfigVisible": true,
    "latencyTableEnabled": false,
    "onlyShowPercent": false,
    "returnCriteria": {
      "metricComparisons": [
        {
          "comparisonType": "ge",
          "metric": {
            "id": "metrics/occurrences", "__entity__": true, "type": "Metric",
            "__metaData__": { "name": "Events" }
          },
          "value": 1
        }
      ],
      "metricOperator": "and",
      "segments": []
    },
    "rollingCohortEnabled": false,
    "roundPercentToNearestWhole": false,
    "showAnnotations": true,
    "showAveragePercentRow": true,
    "showControls": true,
    "type": "CohortReportlet"
  },
  "swatchColor": "#00C0C7",
  "type": "genericSubPanel",
  "visible": true,
  "visualizationIndex": 1
}
```

Key notes:

- `linkedSourceId` must be `""` (empty) — Cohort Table is self-contained
- `inclusionCriteria` and `returnCriteria` can use **different** metrics (e.g., include on "Visits", return on "Events")
- `granularityValue` values are capitalized: `"Day"`, `"Week"`, `"Month"` (not lowercase)
- `churn: false` = retention (who came back); `churn: true` = churn (who did NOT come back)
- `container: "visitors"` tracks at the person level
- Subpanel `autoHeight: 616` is the standard height for cohort tables
- `visualizationIndex: 1` (not 0)

Histogram Visualization

A `HistogramReportlet` shows the distribution of a metric across automatically generated buckets. Unlike most chart types, the histogram configures its own metric via the `component` property. It still requires a linked FreeformReportlet, but the **linked table should be empty and hidden** — the histogram handles all data configuration itself.

Structure

Property	Type	Description
`type`	string	`"HistogramReportlet"`
`component`	Entity	**Required.** The metric entity to create the distribution for
`name`	string	Display name for the histogram (e.g.,`"Events per Visitor"`)
`hasLimit`	boolean	Whether to limit the displayed range
`numItems`	number	Number of items to display (default: 14)
`usePercentage`	boolean	Show values as percentages
`disabledLegends`	array	Legend items to hide
`legendMappings`	LegendSettings	`{ "entries": [] }`
`legendVisible`	boolean	Show the chart legend

Optional advanced properties (the histogram auto-determines these if omitted):

Property	Type	Description
`bucketCount`	number	Number of buckets (e.g., 12)
`bucketSize`	number	Size of each bucket (e.g., 1)
`startingBucket`	number	Starting value for the first bucket (e.g., 1)
`countingMethod`	string	`"hits"`, `"visits"`, `"visitors"`, etc.

Linked Table: Empty and Hidden

The histogram requires a `linkedSourceId` pointing to a FreeformReportlet, but the table should be **empty** (no `nodes` in `columnTree`, no `dimensionSettings`) and **hidden** (`visible: false`). The histogram uses its own `component` property for all data configuration.

```
{
  "reportlet": {
    "type": "FreeformReportlet",
    "columnTree": {
      "id": "CT-ROOT", "name": "", "_computedValues": [], "visible": true,
      "nodes": []
    },
    "freeformTable": {
      "breakdowns": [],
      "columnWidths": [],
      "dimensionSettings": [],
      "pagination": { "currentPage": 0, "viewBy": 10 },
      "sort": { "asc": true, "columnId": "", "labelColumn": true },
      "settings": { "breakdownByPosition": false, "rowBasedPercentages": false, "totalsType": "allVisits" },
      "staticRows": [],
      "statistics": { "functions": [], "ignoreZeros": true }
    }
  }
}
```

Minimal Histogram Reportlet

The histogram reportlet itself is very concise — just `component`, `name`, and a few standard properties:

```
{
  "component": {
    "id": "metrics/occurrences",
    "__entity__": true,
    "type": "Metric",
    "__metaData__": { "name": "Events" }
  },
  "disabledLegends": [],
  "hasLimit": true,
  "legendMappings": { "entries": [] },
  "legendVisible": true,
  "name": "Events per Visitor",
  "numItems": 14,
  "type": "HistogramReportlet",
  "usePercentage": false
}
```

Key notes:

- The histogram requires `linkedSourceId` pointing to a FreeformReportlet, but the table is **empty and hidden**
- The `component` property (metric Entity) drives all data — no dimension or column tree needed
- Optional `bucketCount`, `bucketSize`, `startingBucket`, and `countingMethod` can be omitted — the histogram auto-determines sensible defaults
- `visualizationIndex: 1` for the histogram subpanel

Journey Canvas Visualization

A `JourneyCanvasReportlet` shows a visual journey map — a directed graph of nodes (touchpoints) connected by edges (transitions). Like Fallout, Flow, and Cohort, Journey Canvas is standalone and does **not** use `linkedSourceId`. All configuration lives in the `mfeJson` property, a freeform object representing the React Flow graph.

Structure

Property	Type	Description
`type`	string	`"JourneyCanvasReportlet"`
`mfeJson`	object	**Required.** The micro-frontend graph definition containing `nodes`, `edges`, `filters`, metrics, and viewport state
`hideTitle`	boolean	Hide the visualization title
`isConfigVisible`	boolean	Show the configuration panel
`showAnnotations`	boolean	Show annotation markers
`showControls`	boolean	Show the control bar

mfeJson Object

The `mfeJson` object uses a React Flow graph structure:

Field	Type	Description
`nodes`	array	Array of graph nodes (touchpoints). Each has `id`, `type`, `data`, and `position`.
`edges`	array	Array of directed edges connecting nodes. Each has `id`, `type`, `source`, `target`, `sourceHandle`, `targetHandle`.
`filters`	object	Display settings:`selectedNodeTypes`, `arrowSetting`, `percentageSetting`, `showFallout`
`container`	string	Counting container:`"visitors"`, `"visits"`, etc.
`primaryMetric`	object	Primary metric for the journey:`{ "id": "metrics/visitors", "type": "Metric" }` (simplified entity, no `__entity__`)
`secondaryMetric`	object	Secondary metric:`{ "id": "metrics/occurrences", "type": "Metric" }` (simplified entity, no `__entity__`)
`isSequential`	boolean	`true` = enforce sequential ordering between nodes
`loadFromAjoJourney`	boolean	`false` for custom journeys; `true` to load from Adobe Journey Optimizer
`stateViewport`	object	Viewport position and zoom:`{ "x": number, "y": number, "zoom": number }`

`version`	number	mfeJson format version (current:21)

Node Object

Each node represents a touchpoint in the journey:

```
{
  "id": "node-a",
  "type": "component",
  "data": {
    "type": "Dimension item",
    "iconType": "dimensionItem",
    "components": [
      {
        "id": "variables/daterangeday::1260201",
        "itemType": "dimensionItem",
        "name": "Mar 1, 2026",
        "type": "DimensionItem"
      }
    ]
  },
  "position": { "x": 16, "y": 208 }
}
```

Field	Type	Description
`id`	string	Unique node identifier
`type`	string	`"component"` for standard touchpoint nodes
`data.type`	string	`"Dimension item"` for dimension item nodes
`data.iconType`	string	`"dimensionItem"`
`data.components`	array	Array of component references (simplified — no `__entity__`, uses `itemType` instead)
`position`	object	`{ "x": number, "y": number }` — canvas coordinates. Space nodes ~256px apart horizontally.

Edge Object

Each edge connects two nodes directionally:

```
{
  "id": "xy-edge__node-aright-source-node-bleft-target",
  "type": "journey",
  "source": "node-a",
  "target": "node-b",
  "sourceHandle": "right-source",
  "targetHandle": "left-target",
  "data": {}
}
```

Field	Type	Description
`id`	string	Convention:`"xy-edge__<source-id><sourceHandle>-<target-id><targetHandle>"`
`type`	string	`"journey"`
`source`	string	Source node ID
`target`	string	Target node ID
`sourceHandle`	string	`"right-source"` for left-to-right flow
`targetHandle`	string	`"left-target"` for left-to-right flow
`data`	object	Additional edge data (typically empty `{}`)

Filters Object

```
{
  "filters": {
    "selectedNodeTypes": ["Dimension item"],
    "arrowSetting": "labels",
    "percentageSetting": "percentoftotal",
    "showFallout": false
  }
}
```

Field	Type	Description
`selectedNodeTypes`	array	Node types to display: `"Dimension item"`, `"Metric"`, `"Segment"`, etc.
`arrowSetting`	string	`"labels"` = show labels on arrows, `"none"` = hide
`percentageSetting`	string	`"percentoftotal"` or `"percentofnode"`
`showFallout`	boolean	Show fallout (drop-off) between nodes

Metric References in mfeJson

Unlike the rest of the project definition, metrics inside `mfeJson` use a **simplified entity format** without `__entity__: true` or `__metaData__`:

```
{
  "primaryMetric": { "id": "metrics/visitors", "type": "Metric" },
  "secondaryMetric": { "id": "metrics/occurrences", "type": "Metric" }
}
```

Similarly, node `components` use `itemType` instead of the standard Entity pattern:

```
{
  "id": "variables/daterangeday::1260201",
  "itemType": "dimensionItem",
  "name": "Mar 1, 2026",
  "type": "DimensionItem"
}
```

Complete Journey Canvas Example

A 3-node sequential journey using Day dimension items:

```
{
  "id": "SP-JOURNEY",
  "linkedSourceId": "",
  "position": { "autoHeight": 817, "autoSize": true, "width": 100, "x": 0, "y": 0 },
  "reportlet": {
    "hideTitle": false,
    "intelligentCaptions": { "captions": [], "hasLoaded": false, "hiddenCaptions": [], "isExpanded": false },
    "isConfigVisible": true,
    "mfeJson": {
      "edges": [
        {
          "id": "xy-edge__node-aright-source-node-bleft-target",
          "type": "journey",
          "source": "node-a",
          "target": "node-b",
          "sourceHandle": "right-source",
          "targetHandle": "left-target",
          "data": {}
        },
        {
          "id": "xy-edge__node-bright-source-node-cleft-target",
          "type": "journey",
          "source": "node-b",
          "target": "node-c",
          "sourceHandle": "right-source",
          "targetHandle": "left-target",
          "data": {}
        }
      ],
      "nodes": [
        {
          "id": "node-a",
```

```
          "type": "component",
          "data": {
            "type": "Dimension item",
            "iconType": "dimensionItem",
            "components": [
              { "id": "variables/daterangeday::1260201", "itemType": "dimensionItem", "name": "Mar 1, 2026", "type"
: "DimensionItem" }
            ]
          },
          "position": { "x": 16, "y": 208 }
        },
        {
          "id": "node-b",
          "type": "component",
          "data": {
            "type": "Dimension item",
            "iconType": "dimensionItem",
            "components": [
              { "id": "variables/daterangeday::1260202", "itemType": "dimensionItem", "name": "Mar 2, 2026", "type"
: "DimensionItem" }
            ]
          },
          "position": { "x": 272, "y": 208 }
        },
        {
          "id": "node-c",
          "type": "component",
          "data": {
            "type": "Dimension item",
            "iconType": "dimensionItem",
            "components": [
              { "id": "variables/daterangeday::1260203", "itemType": "dimensionItem", "name": "Mar 3, 2026", "type"
: "DimensionItem" }
            ]
          },
          "position": { "x": 528, "y": 208 }
        }
      ],
      "filters": {
        "selectedNodeTypes": ["Dimension item"],
        "arrowSetting": "labels",
        "percentageSetting": "percentoftotal",
        "showFallout": false
      },
      "container": "visitors",
      "primaryMetric": { "id": "metrics/visitors", "type": "Metric" },
      "secondaryMetric": { "id": "metrics/occurrences", "type": "Metric" },
      "loadFromAjoJourney": false,
      "isSequential": true,
      "stateViewport": { "x": -63, "y": -197, "zoom": 1.23 },
      "version": 21
    },
    "showAnnotations": true,
    "showControls": true,
    "type": "JourneyCanvasReportlet"
  },
  "swatchColor": "#00C0C7",
  "type": "genericSubPanel",
  "visible": true,
  "visualizationIndex": 1
}
```


Key notes:

- `linkedSourceId` must be `""` (empty) — Journey Canvas is self-contained
- `mfeJson` uses a React Flow graph format — not standard CJA Entity objects
- Metrics in `mfeJson` use simplified format: `{ "id": "...", "type": "Metric" }` — no `__entity__` or `__metaData__`
- Node components use `itemType` instead of `__entity__` pattern
- Node `position` is in canvas pixels; space nodes ~256px apart horizontally
- Edge IDs follow the convention `"xy-edge__<sourceId><sourceHandle>-<targetId><targetHandle>"`
- Panel `autoHeight: 991`, subpanel `autoHeight: 817`
- `mfeJson.version: 21` is the current format version
- `visualizationIndex: 1` (not 0)

Key Metric Summary Visualization

A `KeyMetricSummaryReportlet` shows a single metric's value with a trend sparkline and comparison to a previous period. It links to a FreeformReportlet for data and has its own `keyMetric`, `primaryDateRange`, and `comparisonDateRange` properties — all of which must be set on both the reportlet and inside `config` (same pattern as ComboReportlet).

Structure

Property	Type	Description
`type`	string	`"KeyMetricSummaryReportlet"`
`keyMetric`	Entity	**Required.** The metric to display. Must be set on both reportlet and inside `config`.
`primaryDateRange`	Entity	**Required.** Primary date range. DateRange entity with `__metaData__.definition` (e.g., `"tm/tm+1m"`). Must be set on both reportlet and inside `config`.
`comparisonDateRange`	Entity	**Required.** Comparison date range. DateRange entity with `__metaData__.definition` (e.g., `"tm-1m/tm"`). Must be set on both reportlet and inside `config`.
`config`	KeyMetricSummaryConfig	**Required.** Configuration object mirroring `keyMetric`, `primaryDateRange`, `comparisonDateRange`. Must include `"type": "reportletConfig"`.
`displayType`	string	`"change"` — show the change between periods
`showComparison`	boolean	Show the comparison value
`showTrendlines`	boolean	Show the trend sparkline
`summaryDisplayType`	string	`"percent"` — show comparison as percentage change
`showRawDifference`	boolean	Show raw numeric difference (`false`)
`showMax`	boolean	Show max value (`false`)
`showMin`	boolean	Show min value (`false`)
`showMaxMin`	boolean	Show max/min values (`false`)
`showXAxis`	boolean	Show the X-axis on the sparkline (`true`)
`showVizSelectorOnSubPanel`	boolean	Show viz selector (`false`)
`abbreviateValue`	boolean	Abbreviate large numbers (`false`)
`abbreviatePrecision`	number	Decimal precision when abbreviating (`0`)
`anchorToLeft`	boolean	Anchor the sparkline to the left (`false`)
`hasLimit`	boolean	**Required.** `false`
`numItems`	number	**Required.** `14`
`legendVisible`	boolean	`true`
`lockType`	string	`"positions"`
`usePercentage`	boolean	`false`
`useRowBasedPercentages`	boolean	`false`

Important: Do NOT include `legendMappings` on the KMS reportlet — it is not valid for this type and will cause the visualization to be invalid.

Data Table for Key Metric Summary (Critical)

The linked FreeformReportlet must have a **two-column structure** with date ranges matching the KMS's `primaryDateRange` and `comparisonDateRange`. Without this, the reportlets are "out of sync".

Every column node and DateRange child node must include full `dataSettings`, `selectionCoordinates`, and `tableCellDisplay` objects. Minimal nodes will cause validation failures.

```
"nodes": [
  {
    "_computedValues": [], "visible": true,
    "component": { "id": "metrics/occurrences", "__entity__": true, "type": "Metric", "__metaData__": { "name": "E
vents" } },
    "dataSettings": { "advancedItemLimit": 5, "advancedItemSearch": { "alwaysExcludedItems": [], "operator": "AND"
, "rules": [] } },
    "id": "COL-001", "name": "Events",
    "nodes": [
      {
        "_computedValues": [], "visible": true,
        "component": { "__entity__": true, "type": "DateRange", "__metaData__": { "definition": "tm/tm+1m" } },
        "dataSettings": { "advancedItemLimit": 5, "advancedItemSearch": { "alwaysExcludedItems": [], "operator": "
AND", "rules": [] } },
        "id": "COL-001-DR", "name": "This month", "nodes": [],
        "selectionCoordinates": [],
        "tableCellDisplay": {
          "conditionalFormattingOpts": { "autoGenerate": true, "colorPalette": "#ffa48c,#ffcb94,#fff19b,#cadf7d,
#95cc5f", "usePercentLimits": false },
          "location": "behindNumber",
          "type": { "background": true, "backgroundType": "bar", "comparison": "none", "interpretZeroAsNoValue": f
alse, "number": true, "percent": true, "showAnomaly": false, "showForecasting": false, "showGrandTotal": true, "sh
owSparklines": true, "showTotals": true, "wrapHeaderText": true }
        }
      }
    ],
    "selectionCoordinates": [],
    "tableCellDisplay": {
      "conditionalFormattingOpts": { "autoGenerate": true, "colorPalette": "#ffa48c,#ffcb94,#fff19b,#cadf7d,
#95cc5f", "usePercentLimits": false },
      "location": "behindNumber",
      "type": { "background": true, "backgroundType": "bar", "comparison": "none", "interpretZeroAsNoValue": false
, "number": true, "percent": true, "showAnomaly": false, "showForecasting": false, "showGrandTotal": true, "showSp
arklines": true, "showTotals": true, "wrapHeaderText": true }
    }
  },
  {
    "_computedValues": [], "visible": true,
    "component": { "id": "metrics/occurrences", "__entity__": true, "type": "Metric", "__metaData__": { "name": "E
vents" } },
    "dataSettings": { "advancedItemLimit": 5, "advancedItemSearch": { "alwaysExcludedItems": [], "operator": "AND"
, "rules": [] } },
    "id": "COL-002", "name": "Events",
    "nodes": [
      {
        "_computedValues": [], "visible": true,
        "component": { "__entity__": true, "type": "DateRange", "__metaData__": { "definition": "tm-1m/tm" } },
        "dataSettings": { "advancedItemLimit": 5, "advancedItemSearch": { "alwaysExcludedItems": [], "operator": "
AND", "rules": [] } },
        "id": "COL-002-DR", "name": "Previous month", "nodes": [],
        "selectionCoordinates": [],
        "tableCellDisplay": {
          "conditionalFormattingOpts": { "autoGenerate": true, "colorPalette": "#ffa48c,#ffcb94,#fff19b,#cadf7d,
#95cc5f", "usePercentLimits": false },
          "location": "behindNumber",
          "type": { "background": true, "backgroundType": "bar", "comparison": "none", "interpretZeroAsNoValue": f
alse, "number": true, "percent": true, "showAnomaly": false, "showForecasting": false, "showGrandTotal": true, "sh
owSparklines": true, "showTotals": true, "wrapHeaderText": true }
        }
      }
    ],
    "selectionCoordinates": [],
    "tableCellDisplay": {
      "conditionalFormattingOpts": { "autoGenerate": true, "colorPalette": "#ffa48c,#ffcb94,#fff19b,#cadf7d,
#95cc5f", "usePercentLimits": false },
      "location": "behindNumber",
      "type": { "background": true, "backgroundType": "bar", "comparison": "none", "interpretZeroAsNoValue": false
, "number": true, "percent": true, "showAnomaly": false, "showForecasting": false, "showGrandTotal": true, "showSp
arklines": true, "showTotals": true, "wrapHeaderText": true }
    }
  }
]
```


Key difference from ComboReportlet: Both columns have nested DateRange children (the primary period and the comparison period). In the ComboReportlet, only the second column has a nested DateRange.

DateRange definition Syntax

The `primaryDateRange` and `comparisonDateRange` use DateRange entities with `__metaData__.definition` (no `id`):

Definition	Meaning
`tm/tm+1m`	This month (current month)
`tm-1m/tm`	Previous month
`td-29d/td+1d`	Last 30 days
`td-59d/td-29d`	Previous 30 days (before last 30)

Key notes:

- `keyMetric`, `primaryDateRange`, and `comparisonDateRange` must be on both the reportlet AND inside `config`
- `config.type` must be `"reportletConfig"`
- The linked table needs two metric columns, each with a nested DateRange child matching the primary and comparison periods
- Without matching date ranges in the table, CJA shows an "out of sync" warning
- The data table should be `visible: false`
- Do NOT include `legendMappings` on the KMS reportlet — this property is not valid for KMS and causes the visualization to fail
- The KMS reportlet must include `hasLimit: false` and `numItems: 14`
- The backing table **must** include `dimensionSettings` with a Day dimension (or appropriate time dimension) and `search` object
- The backing table's `totalsType` should be `"allVisits"` (not `"columnSum"`)
- The backing table's `sort` should use `labelColumn: true`
- Every column node and DateRange child in the backing table must include full `dataSettings`, `selectionCoordinates`, and `tableCellDisplay` objects — minimal nodes will be invalid
- The `columnTree` root and each node must include `tableCellDisplay` and `selectionCoordinates`
- Subpanel `autoHeight: 477` is standard for KMS; panel `autoHeight: 651`
- `visualizationIndex: 1`

Map Visualization

A `MapReportlet` shows geographic data on an interactive map. Like Fallout, Flow, Cohort, and Journey Canvas, the Map is standalone and does **not** use `linkedSourceId`. It configures its own metric via the `metricComponent` property.

Structure

Property	Type	Description
`type`	string	`"MapReportlet"`
`metricComponent`	Entity	**Required.** The metric entity to visualize on the map
`mapType`	string	`"bubbles"` (sized circles) or `"heatmap"` (heat overlay)
`mapStyle`	string	Map base style:`"light"`, `"dark"`, `"streets"`, etc.
`colorTheme`	string	Color palette for data visualization:`"reds1"`, `"blues1"`, etc.
`isLatLng`	boolean	`true` = use latitude/longitude dimensions; `false` = use geo dimension. **Recommended: `true`**
`latitudeDimension`	Entity	Latitude dimension entity (when `isLatLng: true` with custom lat dimension)
`longitudeDimension`	Entity	Longitude dimension entity (when `isLatLng: true` with custom lng dimension)
`center`	array	Map center as `[latitude, longitude]` (e.g., `[-45.34, 29.3]` for a world view)
`zoom`	number	Zoom level (e.g.,`0.43` for zoomed-out world view, higher = more zoomed in)
`bearing`	number	Map rotation in degrees (typically `0`)
`pitch`	number	Map tilt in degrees (typically `0`)
`clusterRadius`	number	Pixel radius for clustering nearby points (e.g.,`50`)
`customMaxValue`	number	Custom maximum value for the scale (0 = auto)
`useCustomMaxValue`	boolean	Whether to use `customMaxValue`
`hideTitle`	boolean	Hide the visualization title
`isConfigVisible`	boolean	Show the configuration panel
`showAnnotations`	boolean	Show annotation markers
`showControls`	boolean	Show the control bar

Complete Map Example

A bubble map showing Events by geography:

```
{
  "id": "SP-MAP",
  "linkedSourceId": "",
  "position": { "autoHeight": 477, "autoSize": true, "width": 50, "x": 0, "y": 0 },
  "reportlet": {
    "bearing": 0,
    "center": [-45.34, 29.3],
    "clusterRadius": 50,
    "colorTheme": "reds1",
    "customMaxValue": 0,
    "hideTitle": false,
    "intelligentCaptions": { "captions": [], "hasLoaded": false, "hiddenCaptions": [], "isExpanded": false },
    "isConfigVisible": true,
    "isLatLng": true,
    "mapStyle": "light",
    "mapType": "bubbles",
    "metricComponent": {
      "id": "metrics/occurrences",
      "__entity__": true,
      "type": "Metric",
      "__metaData__": { "name": "Events" }
    },
    "pitch": 0,
    "showAnnotations": true,
    "showControls": true,
    "showVizSelectorOnSubPanel": false,
    "type": "MapReportlet",
    "useCustomMaxValue": false,
    "useRowBasedPercentages": false,
    "zoom": 0.43
  },
  "swatchColor": "#2780EB",
  "type": "genericSubPanel",
  "visible": true,
  "visualizationIndex": 1
}
```

Key notes:

- `linkedSourceId` must be `""` (empty) — Map is self-contained
- `metricComponent` (not `component` like Histogram) is the metric entity
- `isLatLng: true` is recommended — uses latitude/longitude for positioning
- `center` is `[latitude, longitude]` (note: latitude first, not longitude)
- `zoom: 0.43` gives a good zoomed-out world view; increase for more focused regions
- `width: 50` makes the map half-width (common layout); use `100` for full width
- Panel `autoHeight: 651`, subpanel `autoHeight: 477`
- `visualizationIndex: 1`

Venn Visualization

The `VennReportlet` shows segment overlap as a Venn diagram. Each segment becomes a circle. Unlike most chart types, the Venn reportlet holds its own `metrics` and `segments` arrays directly on the reportlet (not read from the column tree). It links to a **hidden, empty** `FreeformReportlet`.

Structure

```
{
  "reportlet": {
    "contextForSegmentConversion": "visits",
    "disabledLegends": [],
    "hideTitle": false,
    "intelligentCaptions": {
      "captions": [], "hasLoaded": false, "hiddenCaptions": [], "isExpanded": false
    },
    "isConfigVisible": true,
    "legendMappings": { "entries": [] },
    "legendVisible": true,
    "lockType": "positions",
    "metrics": [
      {
        "id": "metrics/occurrences", "__entity__": true, "type": "Metric",
        "__metaData__": { "name": "Events" }
      }
    ],
    "normalized": false,
    "segments": [
      {
        "id": "All_Visits", "__entity__": true, "type": "Segment",
        "__metaData__": { "name": "All Data" }
      },
      {
        "id": "s_SEGMENT_ID", "__entity__": true, "type": "Segment",
        "__metaData__": { "name": "My Segment" }
      }
    ],
    "showAnnotations": true,
    "showControls": true,
    "showVizSelectorOnSubPanel": false,
    "type": "VennReportlet",
    "useRowBasedPercentages": false
  }
}
```

Venn-Specific Properties

Property	Type	Description
`metrics`	array of Metric entities	Metrics used to calculate overlap (typically one metric, e.g., Events)
`segments`	array of Segment entities	Each segment becomes a circle in the Venn diagram. Use 2–3 segments for a meaningful overlap visualization.
`contextForSegmentConversion`	string	Counting container for the segment overlap:`"visits"`, `"visitors"`
`normalized`	boolean	Whether to normalize values
`showVizSelectorOnSubPanel`	boolean	Typically `false` for Venn

Linked FreeformReportlet

The Venn links to a **hidden, empty** `FreeformReportlet` (same pattern as Histogram):

- `visible: false` — the table is hidden
- `columnTree.nodes: []` — no columns defined
- `name: "Venn Data"` (optional descriptive name)
- The subpanel has `linkedSourceId: ""` (it's the data source)

Key Notes

- Each entry in `segments` becomes one circle in the Venn diagram
- `metrics` is an array but typically contains just one metric
- `"All_Visits"` is the built-in "All Data" segment ID
- Panel `autoHeight: 551`, Venn subpanel `autoHeight: 377`
- The linked table is similar to Histogram's empty hidden table

Summary Change Visualization

A `SummaryChangeReportlet` shows the percent or absolute change between two metric values — typically a metric and its comparison period (e.g., this week vs last week). It reads data from a linked FreeformReportlet and selects **two data points** via `lockedSelection` to compute the change.

Structure

Property	Type	Description
`type`	string	`"SummaryChangeReportlet"`
`displayType`	string	`"change"` — show the change value
`lockedSelection`	FreeformBreakdownSelection	**Required.** Must select exactly **two** `selectedDataPoints` — the current and comparison values. Both typically use `rowPosition: -1` (totals row) with different `columnIds`.
`lockType`	string	`"positions"`
`abbreviateValue`	boolean	Abbreviate large numbers
`abbreviatePrecision`	number	Decimal precision when abbreviating
`usePercentage`	boolean	Show as percentage
`useRowBasedPercentages`	boolean	Use row-based percentages

Complete Example

```
{
  "id": "SP-CHANGE",
  "linkedSourceId": "SP-DATA-TABLE",
  "name": "Events WoW",
  "position": { "autoHeight": 200, "autoSize": false, "fixedHeight": 200, "width": 16.66, "x": 0, "y": 0 },
  "reportlet": {
    "abbreviatePrecision": 0,
    "abbreviateValue": false,
    "displayType": "change",
    "hideTitle": false,
    "intelligentCaptions": { "captions": [], "hasLoaded": false, "hiddenCaptions": [], "isExpanded": false },
    "isConfigVisible": true,
    "legendVisible": true,
    "lockedSelection": {
      "breakdowns": [],
      "selectedDataPoints": [
        { "columnId": "COL-CURRENT", "rowPosition": -1 },
        { "columnId": "COL-COMPARISON", "rowPosition": -1 }
      ],
      "type": "FreeformBreakdownSelection"
    },
    "lockType": "positions",
    "name": "Events WoW",
    "showAnnotations": true,
    "showControls": true,
    "type": "SummaryChangeReportlet",
    "usePercentage": false,
    "useRowBasedPercentages": false
  },
  "type": "genericSubPanel",
  "visible": true,
  "visualizationIndex": 1
}
```

Key notes:

- Requires `linkedSourceId` pointing to a FreeformReportlet with at least two metric columns (current + comparison)
- `lockedSelection` must have exactly **two** `selectedDataPoints` — the first is the current value, the second is the comparison
- Both points typically use `rowPosition: -1` (totals) with different `columnIds` pointing to the two metric columns
- `"type": "FreeformBreakdownSelection"` is required on the `lockedSelection` object
- Common layout: `fixedHeight: 200`, narrow `width: 16.66` (for 6-across tile layout)

Section Header Visualization

A `SectionHeaderReportlet` is a non-data structural element — a labeled divider used to organize long panels into visual sections. It does not display data and does not use `linkedSourceId`.

Structure

Property	Type	Description
`type`	string	`"SectionHeaderReportlet"`
`name`	string	Display text for the section header
`hideTitle`	boolean	`false` to show the header text

Complete Example

```
{
  "id": "SP-HEADER",
  "linkedSourceId": "",
  "name": "Channel Metrics",
  "position": { "autoHeight": 69, "autoSize": true, "width": 100, "x": 0, "y": 0 },
  "reportlet": {
    "hideTitle": false,
    "intelligentCaptions": { "captions": [], "hasLoaded": false, "hiddenCaptions": [], "isExpanded": false },
    "isConfigVisible": true,
    "name": "Channel Metrics",
    "showAnnotations": true,
    "showControls": true,
    "type": "SectionHeaderReportlet",
    "useRowBasedPercentages": false
  },
  "swatchColor": "#FEE02D",
  "type": "genericSubPanel",
  "visible": true,
  "visualizationIndex": 1
}
```

Key notes:

- `linkedSourceId` must be `""` — no data source
- `autoHeight: 69` is the standard height
- `swatchColor` sets the accent color on the left side of the header
- `name` appears on both the subpanel and the reportlet

TextReportlet

The `TextReportlet` displays rich text content using a Quill delta JSON string in `textContent` (same format as subpanel descriptions — see GenericSubPanel section):

```
{
  "type": "TextReportlet",
  "textContent": "{\"ops\":[{\"attributes\":{\"bold\":true},\"insert\":\"Dashboard Summary\"},{\"insert\":\"
\\nThis project tracks weekly conversion metrics.\\n\"}]}",
  "hideTitle": true
}
```

`linkedSourceId` must be `""` — TextReportlet is self-contained and does not read from a data table.

Area and Stacked Chart Variants

These chart types follow the same linked-table pattern as `LineReportlet` and `BarReportlet` — they read data from a sibling FreeformReportlet via `linkedSourceId`.

AreaReportlet

An area chart (filled line). Same properties as `LineReportlet` plus:

Property	Type	Description
`type`	string	`"AreaReportlet"`
`normalized`	boolean	Normalize values

Supports `showAnomalies`, `dualYAxis`, `forceZero`, `anchorToLeft`, `legendMappings`, and all standard chart properties.

AreaStackedReportlet

A stacked area chart — multiple series stacked on top of each other.

Property	Type	Description
`type`	string	`"AreaStackedReportlet"`
`stacked100`	boolean	`true` = normalize stacks to 100%; `false` = absolute values
`forceZero`	boolean	Force Y-axis to start at zero

Supports `showAnomalies`, `anchorToLeft`, `legendMappings`, `lockedSelection`, and all standard chart properties.

BarStackedReportlet

A stacked bar chart — multiple series stacked horizontally.

Property	Type	Description
`type`	string	`"BarStackedReportlet"`
`stacked100`	boolean	`true` = normalize stacks to 100%; `false` = absolute values
`hasLimit`	boolean	Whether to limit displayed items

Supports `legendMappings`, `legendVisible`, `lockType`, `numItems`, and all standard chart properties.

Example (BarStackedReportlet)

```
{
  "id": "SP-STACKED-BAR",
  "linkedSourceId": "SP-DATA-TABLE",
  "position": { "autoHeight": 377, "autoSize": true, "width": 100, "x": 0, "y": 0 },
  "reportlet": {
    "disabledLegends": [],
    "hasLimit": true,
    "hideTitle": false,
    "intelligentCaptions": { "captions": [], "hasLoaded": false, "hiddenCaptions": [], "isExpanded": false },
    "isConfigVisible": true,
    "legendMappings": { "entries": [] },
    "legendVisible": true,
    "lockType": "positions",
    "numItems": 14,
    "showAnnotations": true,
    "showControls": true,
    "stacked100": false,
    "type": "BarStackedReportlet",
    "usePercentage": false,
    "useRowBasedPercentages": false
  },
  "type": "genericSubPanel",
  "visible": true,
  "visualizationIndex": 1
}
```

Scatter Plot Visualization

A `ScatterReportlet` shows a scatter plot with two metrics plotted against each other. Each point represents a dimension item. It reads from a linked FreeformReportlet that has at least two metric columns — the first becomes the X-axis, the second becomes the Y-axis.

Structure

Property	Type	Description
`type`	string	`"ScatterReportlet"`
`forceZero`	boolean	Force axes to start at zero
`hasLimit`	boolean	Limit displayed items
`numItems`	number	Number of dimension items to plot (default: 14)

Complete Example

```
{
  "id": "SP-SCATTER",
  "linkedSourceId": "SP-DATA-TABLE",
  "position": { "autoHeight": 725, "autoSize": false, "fixedHeight": 725, "width": 100, "x": 0, "y": 0 },
  "reportlet": {
    "disabledLegends": [],
    "forceZero": false,
    "hasLimit": false,
    "hideTitle": false,
    "intelligentCaptions": { "captions": [], "hasLoaded": false, "hiddenCaptions": [], "isExpanded": false },
    "isConfigVisible": true,
    "legendMappings": { "entries": [] },
    "legendVisible": true,
    "lockType": "positions",
    "numItems": 14,
    "showAnnotations": true,
    "showControls": true,
    "type": "ScatterReportlet",
    "usePercentage": false,
    "useRowBasedPercentages": false
  },
  "type": "genericSubPanel",
  "visible": true,
  "visualizationIndex": 1
}
```

Key notes:

- The backing FreeformReportlet must have **at least two metric columns** — first metric = X-axis, second = Y-axis
- Each dimension item becomes one point on the scatter
- `fixedHeight: 725` with `autoSize: false` is common for scatter plots (they need vertical space)

Bullet Chart Visualization

A `BulletReportlet` shows a single metric value against a target goal, with low/mid/high thresholds displayed as colored ranges. It reads from a linked FreeformReportlet and uses `lockedSelection` to pick which cell to display.

Structure

Property	Type	Description
`type`	string	`"BulletReportlet"`
`goalVal`	number	**Required.** The target/goal value (displayed as a vertical marker line)
`lowVal`	number	**Required.** Upper bound of the "good" range (green)
`midVal`	number	**Required.** Upper bound of the "satisfactory" range (yellow). Values above `lowVal` and below `midVal` are yellow.
`lockedSelection`	FreeformBreakdownSelection	**Required.** Selects the metric cell to display
`sourceCoords`	string	Typically `""`
`targetCoords`	string	Typically `""`

Complete Example

```
{
  "id": "SP-BULLET",
  "linkedSourceId": "SP-DATA-TABLE",
  "position": { "autoHeight": 300, "autoSize": false, "fixedHeight": 300, "width": 50, "x": 0, "y": 0 },
  "reportlet": {
    "goalVal": 35000,
    "hideTitle": false,
    "intelligentCaptions": { "captions": [], "hasLoaded": false, "hiddenCaptions": [], "isExpanded": false },
    "isConfigVisible": true,
    "legendVisible": true,
    "lockedSelection": {
      "breakdowns": [],
      "selectedDataPoints": [
        { "columnId": "COL-METRIC", "rowPosition": -1 }
      ],
      "type": "FreeformBreakdownSelection"
    },
    "lockType": "positions",
    "lowVal": 45000,
    "midVal": 40000,
    "name": "Orders to Goal",
    "showAnnotations": true,
    "showControls": true,
    "sourceCoords": "",
    "targetCoords": "",
    "type": "BulletReportlet",
    "usePercentage": false,
    "useRowBasedPercentages": false
  },
  "type": "genericSubPanel",
  "visible": true,
  "visualizationIndex": 1
}
```

Key notes:

- The three threshold values define colored ranges: `0goalVal` (red/poor), `goalVallowVal` (yellow/satisfactory), `lowValmidVal` (green/good)
- `lockedSelection` must include `"type": "FreeformBreakdownSelection"` and point to a specific metric column
- `fixedHeight: 300` with `autoSize: false` is standard

Linking Visualizations to Data (linkedSourceId)

Chart visualizations (Line, Bar, Summary Number, etc.) do **not** hold their own data. They read from a sibling FreeformReportlet subpanel via `linkedSourceId`.

How It Works

1. The **data subpanel** (FreeformReportlet) has an `id` and `linkedSourceId: ""`
2. The **chart subpanel** sets `linkedSourceId` to the data subpanel's `id`
3. The chart renders the data from that table

SubPanel Array Order = Visual Order

The order of subPanels in the array determines their visual top-to-bottom rendering order. `visualizationIndex` alone does not control layout position. To place a line chart **above** its data table, the chart subPanel must come **before** the table subPanel in the `subPanels` array:

```
{
  "subPanels": [
    { "id": "CHART-UUID", "linkedSourceId": "TABLE-UUID", "visualizationIndex": 1, "reportlet": { "type": "LineRep
ortlet", ... } },
    { "id": "TABLE-UUID", "linkedSourceId": "", "visualizationIndex": 2, "reportlet": { "type": "FreeformReportlet"
, ... } }
  ]
}
```

If the table comes first in the array, the chart will render **below** the table regardless of `visualizationIndex`.

visualizationIndex Convention

Use **1-based indexing** for `visualizationIndex`. The first subpanel in the array gets `1`, the second gets `2`, etc.

Panel Layout	First SubPanel	Second SubPanel
Chart + Table (linked)	Chart:`visualizationIndex: 1`	Table:`visualizationIndex: 2`
Standalone viz (Fallout, Cohort, etc.)	Viz:`visualizationIndex: 1`	—
Standalone table	Table:`visualizationIndex: 1`	—
Viz + Hidden table (Histogram, Venn)	Viz:`visualizationIndex: 1`	Hidden table:`visualizationIndex: 2`

Example: Freeform Table + Line Chart

```
{
  "subPanels": [
    {
      "id": "DATA-TABLE-UUID",
      "linkedSourceId": "",
      "reportlet": {
        "type": "FreeformReportlet",
        "columnTree": {
          "id": "CT-ROOT",
          "_computedValues": [],
          "visible": true,
          "nodes": [
            {
              "id": "COL-001",
              "_computedValues": [],
              "visible": true,
              "component": { "id": "metrics/visits", "__entity__": true, "type": "Metric" },
              "name": "Visits",
              "nodes": []
            }
          ]
        },
        "freeformTable": {
          "dimensionSettings": [
            {
              "dimension": { "id": "variables/daterangeday", "__entity__": true, "type": "Dimension" },
              "id": "DIM-001"
            }
          ],
          "pagination": { "currentPage": 0, "viewBy": 400 },
          "sort": { "asc": true, "columnId": "", "labelColumn": true },
          "staticRows": [],
          "statistics": { "functions": [], "ignoreZeros": true }
        }
      },
      "type": "genericSubPanel"
    },
    {
      "id": "LINE-CHART-UUID",
      "linkedSourceId": "DATA-TABLE-UUID",
      "reportlet": {
        "type": "LineReportlet",
        "trendlineModel": "linear",
        "dualYAxis": false,
        "showAnomalies": true,
        "showForecast": true,
        "showTrendline": false,
        "forceZero": false,
        "anchorToLeft": false,
        "allowAnomaliesToRescale": false,
        "allowForecastingToRescale": false,
        "movingAverageWindow": 7,
        "legendVisible": true,
        "lockType": "positions",
        "numItems": 14
      },
      "type": "genericSubPanel"
    }
  ]
}
```

Example: Freeform Table + Summary Number

```
{
  "subPanels": [
    {
      "id": "DATA-TABLE-UUID",
      "linkedSourceId": "",
      "reportlet": {
        "type": "FreeformReportlet",
        "columnTree": {
          "id": "CT-ROOT",
          "_computedValues": [],
          "visible": true,
          "nodes": [
            {
              "id": "COL-001",
              "_computedValues": [],
              "visible": true,
              "component": { "id": "metrics/event129", "__entity__": true, "type": "Metric" },
              "name": "Shared Project Load",
              "nodes": []
            }
          ]
        },
        "freeformTable": {
          "dimensionSettings": [
            {
              "dimension": { "id": "variables/daterangeday", "__entity__": true, "type": "Dimension" },
              "id": "DIM-001"
            }
          ],
          "pagination": { "currentPage": 0, "viewBy": 5 },
          "sort": { "asc": true, "columnId": "", "labelColumn": true },
          "staticRows": [],
          "statistics": { "functions": [], "ignoreZeros": true }
        }
      },
      "type": "genericSubPanel"
    },
    {
      "id": "SUMMARY-UUID",
      "linkedSourceId": "DATA-TABLE-UUID",
      "reportlet": {
        "type": "SummaryNumberReportlet",
        "abbreviateValue": false,
        "abbreviatePrecision": 0,
        "summarizeValue": false,
        "summarizeValueBy": "max",
        "usePercentage": false,
        "useRowBasedPercentages": false,
        "lockType": "positions",
        "lockedSelection": {
          "breakdowns": [],
          "selectedDataPoints": [
            { "columnId": "COL-001", "rowPosition": -1 }
          ],
          "type": "FreeformBreakdownSelection"
        },
        "legendVisible": false,
        "sourceCoords": "",
        "isConfigVisible": true
      },
      "type": "genericSubPanel"
    }
  ]
}
```

lockedSelection & FreeformBreakdownSelection (Critical)

The `lockedSelection` property on `SummaryNumberReportlet` (and other chart types) tells the visualization which cell(s) in the source table to display. It uses the `FreeformBreakdownSelection` type.

Critical rule: The `"type": "FreeformBreakdownSelection"` discriminator field is **required** at every level. Without it, the project import fails with: `lockedSelection: None of the provided types can parse this json`.

Structure

```
{
  "breakdowns": [],
  "parentItemIds": [],
  "selectedDataPoints": [
    { "columnId": "<metric-column-id>", "rowPosition": <row-index-or-negative-1> }
  ],
  "type": "FreeformBreakdownSelection"
}
```

Field	Type	Description
`type`	string	**Required.** Always `"FreeformBreakdownSelection"`. Discriminator for the parser.
`selectedDataPoints`	array	Cells to select. Each has `columnId` (metric column ID) and `rowPosition` (0-based index, or `-1` for totals).
`breakdowns`	array	Nested `FreeformBreakdownSelection` entries for breakdown rows. Empty `[]` when selecting top-level rows.
`parentItemIds`	array	Item IDs for item-based locking. Empty `[]` for position-based.
`parentPosition`	number	Used inside `breakdowns` entries to identify which parent row the breakdown belongs to.

Selecting a Top-Level Row

To point a summary number at the totals row (or a specific top-level row):

```
"lockedSelection": {
  "breakdowns": [],
  "selectedDataPoints": [
    { "columnId": "COL-001", "rowPosition": -1 }
  ],
  "type": "FreeformBreakdownSelection"
}
```

`rowPosition: -1` = totals row. `rowPosition: 0` = first row, `1` = second, etc.

Selecting a Breakdown Row (Single Cell)

When the table has breakdowns, the `lockedSelection` must describe the **full breakdown tree** — listing ALL breakdown entries, not just the target. Only the target breakdown has non-empty `selectedDataPoints`; all others have `selectedDataPoints: []`.

The top-level `selectedDataPoints` must be empty `[]` — otherwise you select 2 cells (one at the parent level, one in the breakdown).

Example: Select row 0 of the first breakdown (parentPosition 0), in a table with 3 breakdowns:

```
"lockedSelection": {
  "breakdowns": [
    {
      "breakdowns": [],
      "parentItemIds": [],
      "parentPosition": 0,
      "selectedDataPoints": [
        { "columnId": "COL-001", "rowPosition": 0 }
      ],
      "type": "FreeformBreakdownSelection"
    },
    {
      "breakdowns": [],
      "parentItemIds": [],
      "parentPosition": 1,
      "selectedDataPoints": [],
      "type": "FreeformBreakdownSelection"
    },
    {
      "breakdowns": [],
      "parentItemIds": [],
      "parentPosition": 2,
      "selectedDataPoints": [],
      "type": "FreeformBreakdownSelection"
    }
  ],
  "parentItemIds": [],
  "selectedDataPoints": [],
  "type": "FreeformBreakdownSelection"
}
```

Key rules:

- Top-level `selectedDataPoints: []` (empty) — do not select at the parent level
- `breakdowns` lists ALL breakdown entries (one per `parentPosition`), not just the target
- Only the target breakdown has `selectedDataPoints` with a value; others are `[]`
- Every `FreeformBreakdownSelection` object must have `"type": "FreeformBreakdownSelection"`
- Include `parentItemIds: []` at all levels

Common Patterns

1. Simple Table (One Metric, One Dimension)

A basic "Events by Day" table:

```
{
  "version": "96",
  "workspaces": [
    {
      "id": "WS-UUID",
      "name": "",
      "panels": [
        {
          "id": "PANEL-UUID",
          "name": "Panel",
          "dateRange": { "__entity__": true, "id": "thisMonth", "type": "DateRange" },
          "reportSuite": { "__entity__": true, "id": "dv_YOUR_DATA_VIEW_ID", "type": "ReportSuite" },
          "segmentGroups": [],
          "subPanels": [
            {
              "id": "SP-UUID",
              "linkedSourceId": "",
              "reportlet": {
                "type": "FreeformReportlet",
                "columnTree": {
                  "id": "CT-ROOT",
                  "_computedValues": [],
                  "visible": true,
                  "nodes": [
                    {
                      "id": "COL-001",
                      "_computedValues": [],
                      "visible": true,
                      "component": { "id": "metrics/occurrences", "__entity__": true, "type": "Metric" },
                      "name": "Events",
                      "nodes": []
                    }
                  ]
                },
                "freeformTable": {
                  "dimensionSettings": [
                    {
                      "dimension": { "id": "variables/daterangeday", "__entity__": true, "type": "Dimension" },
                      "id": "DIM-001"
                    }
                  ],
                  "pagination": { "currentPage": 0, "viewBy": 400 },
                  "sort": { "asc": true, "columnId": "", "labelColumn": true },
                  "settings": {
                    "breakdownByPosition": false,
                    "rowBasedPercentages": false,
                    "totalsType": "allVisits"
                  },
                  "staticRows": [],
                  "statistics": { "functions": [], "ignoreZeros": true }
                }
              },
              "type": "genericSubPanel"
            }
          ],
          "type": "panel"
        }
      ]
    }
  ]
}
```

To customize: Replace the `component` entity and `dimension` entity. Add more columns by adding nodes to `columnTree.nodes`. Change `sort.columnId` to a column node ID and `labelColumn: false` to sort by metric.

2. Table with Multiple Metrics

Add more nodes to `columnTree.nodes`:

```
{
  "columnTree": {
    "id": "CT-ROOT",
    "_computedValues": [],
    "visible": true,
    "nodes": [
      {
        "id": "COL-001",
        "_computedValues": [],
        "visible": true,
        "component": { "id": "metrics/visits", "__entity__": true, "type": "Metric" },
        "name": "Visits",
        "nodes": []
      },
      {
        "id": "COL-002",
        "_computedValues": [],
        "visible": true,
        "component": { "id": "metrics/visitors", "__entity__": true, "type": "Metric" },
        "name": "People",
        "nodes": []
      },
      {
        "id": "COL-003",
        "_computedValues": [],
        "visible": true,
        "component": { "id": "metrics/occurrences", "__entity__": true, "type": "Metric" },
        "name": "Events",
        "nodes": []
      }
    ]
  }
}
```

3. Table + Line Chart (Linked)

See the full example in the "Linking Visualizations to Data" section above.

To swap chart type: Change `"type": "LineReportlet"` to `"DonutReportlet"`, `"BarReportlet"`, or `"HorizontalBarReportlet"`. Chart-specific properties (`showAnomalies`, `showForecast`, `showTrendline`) are ignored by non-Line types. To hide the table, set `"visible": false` on the table subpanel.

4. Table with Date Range Comparison Columns

Nest date range entities inside a metric's `nodes` to compare periods. There are two approaches:

Approach A — Saved DateRange IDs (UUIDs): Nest saved DateRange children under a single metric column. Each DateRange child re-nests the metric. This creates sub-columns under one header.

```
{
  "columnTree": {
    "nodes": [
      {
        "component": { "id": "metrics/occurrences", "__entity__": true, "type": "Metric" },
        "name": "Events",
        "nodes": [
          {
            "component": { "id": "66a1299dd667dd2024c72539", "__entity__": true, "type": "DateRange" },
            "name": "May 1 - May 31, 2024",
            "nodes": [
              {
                "component": { "id": "metrics/occurrences", "__entity__": true, "type": "Metric" },
                "name": "Events",
                "nodes": []
              }
            ]
          },
          {
            "component": { "id": "66a12993ba7f4601586b2474", "__entity__": true, "type": "DateRange" },
            "name": "Jun 1 - Jun 30, 2024",
            "nodes": [
              {
                "component": { "id": "metrics/occurrences", "__entity__": true, "type": "Metric" },
                "name": "Events",
                "nodes": []
              }
            ]
          }
        ]
      }
    ]
  }
}
```

Approach B — Rolling formula DateRange (no UUID needed): Use two separate metric columns. The first uses the panel's date range. The second has a DateRange child with a `__metaData__.definition` formula for the comparison period. The DateRange entity has **no `id`** — only `__entity__`, `type`, and `__metaData__.definition`.

Column tree structure:

```
CT-ROOT
 COL-001: Events (uses panel date range — no children)
 COL-002: Events
     COL-002-DR: DateRange (__metaData__.definition: "tm-1m/tm")
```

Full example with all required node properties (`dataSettings`, `selectionCoordinates`, `tableCellDisplay`):

```
{
  "columnTree": {
    "_computedValues": [],
    "dataSettings": {
      "advancedItemLimit": 5,
      "advancedItemSearch": { "alwaysExcludedItems": [], "operator": "AND", "rules": [] }
    },
    "id": "CT-ROOT",
    "name": "",
    "nodes": [
      {
        "_computedValues": [],
        "component": {
          "id": "metrics/occurrences", "__entity__": true, "type": "Metric",
          "__metaData__": { "name": "Events" }
        },
        "dataSettings": {
          "advancedItemLimit": 5,
          "advancedItemSearch": { "alwaysExcludedItems": [], "operator": "AND", "rules": [] }
        },
        "id": "COL-001",
        "name": "Events",
        "nodes": [],
        "selectionCoordinates": [],
        "tableCellDisplay": {
          "conditionalFormattingOpts": {
            "autoGenerate": true,
            "colorPalette": "#ffa48c,#ffcb94,#fff19b,#cadf7d,#95cc5f",
            "usePercentLimits": false
          },
```

```
      "location": "behindNumber",
      "type": {
        "background": true, "backgroundType": "bar", "comparison": "none",
        "interpretZeroAsNoValue": false, "number": true, "percent": true,
        "showAnomaly": false, "showForecasting": false, "showGrandTotal": true,
        "showSparklines": true, "showTotals": true, "wrapHeaderText": true
      }
    },
    "visible": true
  },
  {
    "_computedValues": [],
    "component": {
      "id": "metrics/occurrences", "__entity__": true, "type": "Metric",
      "__metaData__": { "name": "Events" }
    },
    "dataSettings": {
      "advancedItemLimit": 5,
      "advancedItemSearch": { "alwaysExcludedItems": [], "operator": "AND", "rules": [] }
    },
    "id": "COL-002",
    "name": "Events",
    "nodes": [
      {
        "_computedValues": [],
        "component": {
          "__entity__": true, "type": "DateRange",
          "__metaData__": { "definition": "tm-1m/tm" }
        },
        "dataSettings": {
          "advancedItemLimit": 5,
          "advancedItemSearch": { "alwaysExcludedItems": [], "operator": "AND", "rules": [] }
        },
        "id": "COL-002-DR",
        "name": "Previous month",
        "nodes": [],
        "selectionCoordinates": [],
        "tableCellDisplay": {
          "conditionalFormattingOpts": {
            "autoGenerate": true,
            "colorPalette": "#ffa48c,#ffcb94,#fff19b,#cadf7d,#95cc5f",
            "usePercentLimits": false
          },
          "location": "behindNumber",
          "type": {
            "background": true, "backgroundType": "bar", "comparison": "none",
            "interpretZeroAsNoValue": false, "number": true, "percent": true,
            "showAnomaly": false, "showForecasting": false, "showGrandTotal": true,
            "showSparklines": true, "showTotals": true, "wrapHeaderText": true
          }
        },
        "visible": true
      }
    ],
    "selectionCoordinates": [],
    "tableCellDisplay": {
      "conditionalFormattingOpts": {
        "autoGenerate": true,
        "colorPalette": "#ffa48c,#ffcb94,#fff19b,#cadf7d,#95cc5f",
        "usePercentLimits": false
      },
      "location": "behindNumber",
      "type": {
        "background": true, "backgroundType": "bar", "comparison": "none",
        "interpretZeroAsNoValue": false, "number": true, "percent": true,
        "showAnomaly": false, "showForecasting": false, "showGrandTotal": true,
        "showSparklines": true, "showTotals": true, "wrapHeaderText": true
      }
    },
    "visible": true
  }
],
"selectionCoordinates": [],
"tableCellDisplay": {
  "conditionalFormattingOpts": {
    "autoGenerate": true,
    "colorPalette": "#ffa48c,#ffcb94,#fff19b,#cadf7d,#95cc5f",
    "usePercentLimits": false
  },
  "location": "behindNumber",
  "type": {
    "background": true, "backgroundType": "bar", "comparison": "none",
    "interpretZeroAsNoValue": false, "number": true, "percent": true,
    "showAnomaly": false, "showForecasting": false, "showGrandTotal": true,
```

```
        "showSparklines": true, "showTotals": true, "wrapHeaderText": true
      }
    },
    "visible": true
  }
}
```

Key differences between Approach A and B:

	Approach A (Saved UUIDs)	Approach B (Rolling Formula)
DateRange entity	Has `id` (UUID of saved range)	No `id` — uses `__metaData__.definition`
Column structure	Single metric parent with DateRange children, each re-nesting the metric	Two separate metric columns; the comparison column has a DateRange child
Use case	Compare specific fixed periods (e.g., May vs June 2024)	Compare relative/rolling periods (e.g., current month vs previous month)
Requires saved DateRange?	Yes	No

> ***Note***: *For combo charts, the columnTree always needs* ***two separate metric column nodes*** *(Approach B structure) — see the ComboReportlet section for details.*

5. Bar Chart Linked to a Table

```
{
  "subPanels": [
    {
      "id": "TABLE-UUID",
      "linkedSourceId": "",
      "reportlet": {
        "type": "FreeformReportlet",
        "columnTree": {
          "id": "CT-ROOT",
          "_computedValues": [],
          "visible": true,
          "nodes": [
            {
              "id": "COL-001",
              "_computedValues": [],
              "visible": true,
              "component": { "id": "metrics/visits", "__entity__": true, "type": "Metric" },
              "name": "Visits",
              "nodes": []
            }
          ]
        },
        "freeformTable": {
          "dimensionSettings": [
            {
              "dimension": { "id": "variables/mobiledevicename", "__entity__": true, "type": "Dimension" },
              "id": "DIM-001"
            }
          ],
          "pagination": { "currentPage": 0, "viewBy": 5 },
          "sort": { "asc": false, "columnId": "COL-001", "labelColumn": false },
          "staticRows": [],
          "statistics": { "functions": [], "ignoreZeros": true }
        }
      },
      "type": "genericSubPanel"
    },
    {
      "id": "BAR-UUID",
      "linkedSourceId": "TABLE-UUID",
      "position": { "autoHeight": 377, "autoSize": true, "width": 100, "x": 0, "y": 0 },
      "reportlet": {
        "type": "BarReportlet",
        "disabledLegends": [],
        "dualYAxis": false,
        "hasLimit": false,
        "hideTitle": false,
        "intelligentCaptions": { "captions": [], "hasLoaded": false, "hiddenCaptions": [], "isExpanded": false },
        "isConfigVisible": true,
        "legendMappings": { "entries": [] },
        "legendVisible": true,
        "lockType": "positions",
        "normalized": false,
        "numItems": 14,
        "showAnnotations": true,
        "showControls": true,
        "showVizSelectorOnSubPanel": false,
        "usePercentage": false,
        "useRowBasedPercentages": false
      },
      "type": "genericSubPanel"
    }
  ]
}
```

> ***Note**: The bar chart subpanel must come **before** the table in the `subPanels` array to render above it (see "SubPanel Array Order = Visual Order"). The table subpanel's `position.y` should be offset by the bar chart's height (e.g., `y: 377`).*

6. Flat Table with Multiple Dimensions + Bar Chart

A bar chart above a flat table with Day and Hour dimensions. The table has 3 `columnWidths` entries (2 dimension labels + 1 metric) and a dimension sort rule:

```
{
  "subPanels": [
    {
      "id": "SP-BAR",
      "linkedSourceId": "SP-TABLE",
      "position": { "autoHeight": 377, "autoSize": true, "width": 100, "x": 0, "y": 0 },
      "reportlet": {
        "type": "BarReportlet",
        "disabledLegends": [],
        "dualYAxis": false,
        "hasLimit": false,
        "hideTitle": false,
        "intelligentCaptions": { "captions": [], "hasLoaded": false, "hiddenCaptions": [], "isExpanded": false },
        "isConfigVisible": true,
        "legendMappings": { "entries": [] },
        "legendVisible": true,
        "lockType": "positions",
        "normalized": false,
        "numItems": 14,
        "showAnnotations": true,
        "showControls": true,
        "showVizSelectorOnSubPanel": false,
        "usePercentage": false,
        "useRowBasedPercentages": false
      },
      "type": "genericSubPanel",
      "visible": true,
      "visualizationIndex": 0
    },
    {
      "id": "SP-TABLE",
      "linkedSourceId": "",
      "position": { "autoHeight": 13395, "autoSize": true, "width": 100, "x": 0, "y": 377 },
      "reportlet": {
        "type": "FreeformReportlet",
        "columnTree": {
          "id": "CT-ROOT", "_computedValues": [], "visible": true,
          "nodes": [
            {
              "id": "COL-001", "_computedValues": [], "visible": true,
              "component": { "id": "metrics/occurrences", "__entity__": true, "type": "Metric" },
              "name": "Events", "nodes": []
            }
          ]
        },
        "freeformTable": {
          "dimensionSettings": [
            {
              "dimension": { "id": "variables/daterangeday", "__entity__": true, "type": "Dimension" },
              "id": "DIM-001"
            },
            {
              "dimension": { "id": "variables/daterangehour", "__entity__": true, "type": "Dimension" },
              "id": "DIM-002"
            }
          ],
          "columnWidths": [33.33, 33.33, 33.33],
          "breakdowns": [],
          "pagination": { "currentPage": 0, "viewBy": 400 },
          "sort": {
            "advancedSortRules": [
              { "asc": true, "columnId": "DIM-001", "componentType": "dimension" }
            ],
            "asc": true, "columnId": "", "labelColumn": true
          },
          "settings": { "breakdownByPosition": false, "rowBasedPercentages": false, "totalsType": "allVisits" },
          "staticRows": [],
          "statistics": { "functions": [], "ignoreZeros": true }
        }
      },
      "type": "genericSubPanel",
      "visible": true,
      "visualizationIndex": 1
    }
  ]
}
```


Key points: `columnWidths` must have N dimensions + N metrics entries. Table `position.y` is offset by the bar chart height. `advancedSortRules` with `componentType: "dimension"` ensures proper ordering.

7. Panel with Dynamic Dropdown Filter

Add a `segmentGroups` entry to the panel to create an interactive dropdown. Use `runAs: "breakdown"` for a breakdown filter, or omit `runAs` for a segment filter:

```
{
  "segmentGroups": [
    {
      "dynamicDimension": {
        "id": "variables/daterangeday",
        "__entity__": true,
        "type": "Dimension",
        "__metaData__": { "name": "Day" }
      },
      "groupName": "",
      "hasNoFilter": true,
      "selectedComponentOptions": [],
      "showGroupName": true
    }
  ],
  "subPanels": [
    { "...": "bar chart + table subpanels as above" }
  ]
}
```

To make it a breakdown dropdown, add `"runAs": "breakdown"` to the group object. Without `runAs`, it defaults to segment filtering.

8. Fallout Visualization

See the dedicated "Fallout Visualization" section above for the full structure. A minimal subPanels array with a Fallout:

```
{
  "subPanels": [
    {
      "id": "SP-FALLOUT",
      "linkedSourceId": "",
      "position": { "autoHeight": 295, "autoSize": true, "width": 100, "x": 0, "y": 0 },
      "reportlet": {
        "type": "Fallout",
        "checkpoints": [
          {
            "items": [
              { "id": "metrics/occurrences", "__entity__": true, "type": "Metric", "__metaData__": { "name": "Even
ts" } }
            ],
            "name": "", "within": "eventual"
          },
          {
            "items": [
              { "id": "metrics/occurrences", "__entity__": true, "type": "Metric", "__metaData__": { "name": "Even
ts" } }
            ],
            "name": "", "within": "eventual"
          }
        ],
        "falloutContainer": "visitors",
        "includeAllVisitsCheckpoint": true,
        "isConfigVisible": true,
        "legendMappings": { "entries": [] },
        "segments": [
          { "id": "All_Visits", "__entity__": true, "type": "Segment", "__metaData__": { "name": "All Visits" } }
        ],
        "showAnnotations": true,
        "showControls": true
      },
      "swatchColor": "#ED6332",
      "type": "genericSubPanel",
      "visible": true,
      "visualizationIndex": 0
    }
  ]
}
```

Key notes: Fallout uses `linkedSourceId: ""` (self-contained). Add more `checkpoints` entries for additional funnel steps. Use `"falloutContainer": "visitors"` for people or `"visits"` for sessions.

9. Flow Visualization

See the dedicated "Flow Visualization" section above for the full structure. A minimal subPanels array with an entry Flow starting from Day:

```
{
  "subPanels": [
    {
      "id": "SP-FLOW",
      "linkedSourceId": "",
      "position": { "autoHeight": 657, "autoSize": true, "width": 100, "x": 0, "y": 0 },
      "reportlet": {
        "config": {
          "disableTruncation": true,
          "entryComponent": {
            "id": "variables/daterangeday",
            "__entity__": true,
            "type": "Dimension",
            "__metaData__": { "name": "Day" }
          },
          "flowContainer": "visitors",
          "includeRepeatInstances": false,
          "limitEntryExit": true,
          "numColumns": 3,
          "numNodes": 5,
          "type": "reportletConfig"
        },
        "disableTruncation": true,
        "flowContainer": "visitors",
        "includeRepeatInstances": false,
        "isConfigVisible": true,
        "numNodes": 5,
        "rootComponent": {
          "id": "variables/daterangeday",
          "__entity__": true,
          "type": "Dimension",
          "__metaData__": { "name": "Day" }
        },
        "rootPrefixColumn": {
          "childColumn": {
            "childColumn": {
              "dimension": { "id": "variables/daterangeday", "__entity__": true, "type": "Dimension" },
              "expandedNodes": [],
              "limit": 5,
              "search": { "alwaysExcludedItems": [], "operator": "AND", "rules": [] }
            },
            "dimension": { "id": "variables/daterangeday", "__entity__": true, "type": "Dimension" },
            "expandedNodes": [
              { "component": { "id": "variables/daterangeday::1260201", "__entity__": true, "type": "DimensionItem"
, "__metaData__": { "name": "Mar 1, 2026" } } },
              { "component": { "id": "variables/daterangeday::1260202", "__entity__": true, "type": "DimensionItem"
, "__metaData__": { "name": "Mar 2, 2026" } } }
            ],
            "limit": 5,
            "search": { "alwaysExcludedItems": [], "operator": "AND", "rules": [] }
          },
          "dimension": { "id": "variables/daterangeday", "__entity__": true, "type": "Dimension" },
          "expandedNodes": [
            { "component": { "id": "variables/daterangeday", "__entity__": true, "type": "Dimension", "__metaData_
_": { "name": "Day" } } }
          ],
          "limit": 5,
          "search": { "alwaysExcludedItems": [], "operator": "AND", "rules": [] }
        },
        "showAnnotations": true,
        "showControls": true,
        "showGrayPopovers": true,
        "type": "Flow"
      },
      "swatchColor": "#00C0C7",
      "type": "genericSubPanel",
      "visible": true,
      "visualizationIndex": 1
    }
  ]
}
```

Key notes: Flow uses `linkedSourceId: ""` (self-contained). The `config` object with `numColumns` and `entryComponent` is required. The `rootPrefixColumn` must have nested `childColumn` chains matching `numColumns` depth. Level 1 `expandedNodes` uses a Dimension entity; Level 2 uses DimensionItem entities with real IDs — without these, the flow won't expand to all columns. Panel `autoHeight: 831`, subpanel `autoHeight: 657`.

10. Combo Chart with Time Comparison

See the dedicated "ComboReportlet" section above for full details. A combo chart comparing Events this month vs previous month:

```
{
  "subPanels": [
    {
      "id": "SP-COMBO",
      "linkedSourceId": "SP-TABLE",
      "position": { "autoHeight": 377, "autoSize": true, "width": 100, "x": 0, "y": 0 },
      "reportlet": {
        "barMetric": {
          "id": "metrics/occurrences", "__entity__": true, "type": "Metric",
          "__metaData__": { "name": "Events" }
        },
        "barMetricFilters": {
          "first": { "logicalOperator": "and", "show": false },
          "hideFirstItem": true,
          "second": { "logicalOperator": "and", "show": false },
          "third": { "logicalOperator": "and", "show": false }
        },
        "config": {
          "barMetric": {
            "id": "metrics/occurrences", "__entity__": true, "type": "Metric",
            "__metaData__": { "name": "Events" }
          },
          "barMetricFilters": {
            "first": { "logicalOperator": "and", "show": false },
            "hideFirstItem": true,
            "second": { "logicalOperator": "and", "show": false },
            "third": { "logicalOperator": "and", "show": false }
          },
          "dimension": {
            "id": "variables/daterangeday", "__entity__": true, "type": "Dimension",
            "__metaData__": { "name": "Day" }
          },
          "lineComparisons": [
            {
              "comparisonFilters": {
                "first": { "logicalOperator": "and", "show": true },
                "hideFirstItem": true,
                "second": { "logicalOperator": "and", "show": false },
                "third": { "logicalOperator": "and", "show": false }
              },
              "comparisonType": "time-comparison",
              "comparisonValue": {
                "__entity__": true, "type": "DateRange",
                "__metaData__": { "name": "Previous month", "definition": "tm-1m/tm" }
              },
              "id": "LINE-CMP-001"
            }
          ],
          "type": "reportletConfig"
        },
        "dimension": {
          "id": "variables/daterangeday", "__entity__": true, "type": "Dimension",
          "__metaData__": { "name": "Day" }
        },
        "disabledLegends": [],
        "dualYAxis": false,
        "forceZero": true,
        "hasLimit": false,
        "hideTitle": false,
        "intelligentCaptions": { "captions": [], "hasLoaded": false, "hiddenCaptions": [], "isExpanded": false },
        "isConfigVisible": true,
        "legendMappings": { "entries": [] },
        "legendVisible": true,
        "lineComparisons": [
          {
            "comparisonFilters": {
              "first": { "logicalOperator": "and", "show": true },
              "hideFirstItem": true,
              "second": { "logicalOperator": "and", "show": false },
              "third": { "logicalOperator": "and", "show": false }
            },
            "comparisonType": "time-comparison",
            "comparisonValue": {
              "__entity__": true, "type": "DateRange",
              "__metaData__": { "name": "Previous month", "definition": "tm-1m/tm" }
            },
            "id": "LINE-CMP-001"
          }
        ],
        "lockType": "positions",
```

```
        "movingAverageWindow": 7,
        "normalized": false,
        "numItems": 14,
        "showAnnotations": true,
        "showAnomalies": false,
        "showBarbells": true,
        "showControls": true,
        "showMax": false,
        "showMin": false,
        "showTrendline": false,
        "showVizSelectorOnSubPanel": false,
        "showXAxis": true,
        "showYAxis": true,
        "trendlineModel": "linear",
        "type": "ComboReportlet",
        "usePercentage": false,
        "useRowBasedPercentages": false
      },
      "type": "genericSubPanel",
      "visible": true,
      "visualizationIndex": 1
    },
    {
      "id": "SP-TABLE",
      "linkedSourceId": "",
      "name": "Combo Data",
      "position": { "autoHeight": 1261, "autoSize": true, "width": 100, "x": 0, "y": 377 },
      "reportlet": {
        "type": "FreeformReportlet",
        "columnTree": {
          "id": "CT-ROOT", "name": "", "_computedValues": [], "visible": true,
          "nodes": [
            {
              "id": "COL-001", "_computedValues": [], "visible": true,
              "component": { "id": "metrics/occurrences", "__entity__": true, "type": "Metric", "__metaData__": {
"name": "Events" } },
              "name": "Events", "nodes": []
            },
            {
              "id": "COL-002", "_computedValues": [], "visible": true,
              "component": { "id": "metrics/occurrences", "__entity__": true, "type": "Metric", "__metaData__": {
"name": "Events" } },
              "name": "Events",
              "nodes": [
                {
                  "id": "COL-003", "_computedValues": [], "visible": true,
                  "component": { "__entity__": true, "type": "DateRange", "__metaData__": { "name": "Previous
month", "definition": "tm-1m/tm" } },
                  "name": "Previous month", "nodes": []
                }
              ]
            }
          ]
        },
        "freeformTable": {
          "alignDatesForTimeDimension": true,
          "breakdowns": [],
          "columnWidths": [33.33, 33.33, 33.33],
          "dimensionSettings": [
            {
              "dimension": { "id": "variables/daterangeday", "__entity__": true, "type": "Dimension" },
              "id": "DIM-001"
            }
          ],
          "pagination": { "currentPage": 0, "viewBy": 400 },
          "sort": { "asc": true, "columnId": "", "labelColumn": true },
          "settings": { "breakdownByPosition": false, "rowBasedPercentages": false, "totalsType": "allVisits" },
          "staticRows": [],
          "statistics": { "functions": [], "ignoreZeros": true }
        },
        "hideTitle": false,
        "isConfigVisible": true,
        "showAnnotations": true,
        "showControls": true
      },
      "type": "genericSubPanel",
      "visible": false,
      "visualizationIndex": 1
    }
  ]
}
```

Key notes: `barMetric`, `dimension`, and `lineComparisons` must be set on both the reportlet AND inside `config`. The comparison DateRange uses `__metaData__.definition` (e.g., `"tm-1m/tm"`) instead of an `id`. The data table has two separate metric columns (the second with a nested DateRange child), `columnWidths: [33.33, 33.33, 33.33]`, and `visible: false`.

11. Cohort Table

See the dedicated "Cohort Table Visualization" section above for the full structure. A minimal subPanels array with a cohort table tracking Events retention by Day:

```
{
  "subPanels": [
    {
      "id": "SP-COHORT",
      "linkedSourceId": "",
      "position": { "autoHeight": 616, "autoSize": true, "width": 100, "x": 0, "y": 0 },
      "reportlet": {
        "churn": false,
        "cohortTable": {
          "columnWidths": [11.11, 11.11, 11.11, 11.11, 11.11, 11.11, 11.11, 11.11, 11.11],
          "selectionCoordinates": []
        },
        "container": "visitors",
        "dimensionColumnEnabled": false,
        "dimensionSearch": { "operator": "AND", "rules": [] },
        "granularityValue": "Day",
        "inclusionCriteria": {
          "metricComparisons": [
            {
              "comparisonType": "ge",
              "metric": { "id": "metrics/occurrences", "__entity__": true, "type": "Metric", "__metaData__": { "na
me": "Events" } },
              "value": 1
            }
          ],
          "metricOperator": "and",
          "segments": []
        },
        "isConfigVisible": true,
        "latencyTableEnabled": false,
        "onlyShowPercent": false,
        "returnCriteria": {
          "metricComparisons": [
            {
              "comparisonType": "ge",
              "metric": { "id": "metrics/occurrences", "__entity__": true, "type": "Metric", "__metaData__": { "na
me": "Events" } },
              "value": 1
            }
          ],
          "metricOperator": "and",
          "segments": []
        },
        "rollingCohortEnabled": false,
        "roundPercentToNearestWhole": false,
        "showAnnotations": true,
        "showAveragePercentRow": true,
        "showControls": true,
        "type": "CohortReportlet"
      },
      "swatchColor": "#00C0C7",
      "type": "genericSubPanel",
      "visible": true,
      "visualizationIndex": 1
    }
  ]
}
```

Key notes: Cohort uses `linkedSourceId: ""` (self-contained). `inclusionCriteria` and `returnCriteria` each contain `metricComparisons` wit h a metric entity, comparison type, and threshold value. `granularityValue` must be capitalized (`"Day"`, not `"day"`). `churn: false` for retention, `true` for churn analysis.

12. Histogram

See the dedicated "Histogram Visualization" section above for full details. A histogram showing Events distribution, linked to an empty hidden table:

```
{
  "subPanels": [
    {
      "id": "SP-HISTOGRAM",
      "linkedSourceId": "SP-TABLE",
      "position": { "autoHeight": 377, "autoSize": true, "width": 100, "x": 0, "y": 0 },
      "reportlet": {
        "component": {
          "id": "metrics/occurrences", "__entity__": true, "type": "Metric",
          "__metaData__": { "name": "Events" }
        },
        "disabledLegends": [],
        "hasLimit": true,
        "legendMappings": { "entries": [] },
        "legendVisible": true,
        "name": "Events per Visitor",
        "numItems": 14,
        "type": "HistogramReportlet",
        "usePercentage": false
      },
      "swatchColor": "#7E84FA",
      "type": "genericSubPanel",
      "visible": true,
      "visualizationIndex": 1
    },
    {
      "id": "SP-TABLE",
      "linkedSourceId": "",
      "position": { "autoHeight": 377, "autoSize": true, "width": 100, "x": 0, "y": 377 },
      "reportlet": {
        "type": "FreeformReportlet",
        "columnTree": {
          "id": "CT-ROOT", "name": "", "_computedValues": [], "visible": true,
          "nodes": []
        },
        "freeformTable": {
          "breakdowns": [],
          "columnWidths": [],
          "dimensionSettings": [],
          "pagination": { "currentPage": 0, "viewBy": 10 },
          "sort": { "asc": true, "columnId": "", "labelColumn": true },
          "settings": { "breakdownByPosition": false, "rowBasedPercentages": false, "totalsType": "allVisits" },
          "staticRows": [],
          "statistics": { "functions": [], "ignoreZeros": true }
        }
      },
      "type": "genericSubPanel",
      "visible": false,
      "visualizationIndex": 1
    }
  ]
}
```

Key notes: The histogram uses `linkedSourceId` pointing to a FreeformReportlet, but the table is **empty** (no nodes, no dimensionSettings) and **hidden** (`visible: false`). The histogram's `component` property (metric Entity) drives all data configuration. Optional bucketing properties (`bucketCount`, `bucketSize`, `startingBucket`) can be omitted for auto-defaults.

13. Journey Canvas

See the dedicated "Journey Canvas Visualization" section above for full details. A 3-node sequential journey:

```
{
  "subPanels": [
    {
      "id": "SP-JOURNEY",
      "linkedSourceId": "",
      "position": { "autoHeight": 817, "autoSize": true, "width": 100, "x": 0, "y": 0 },
      "reportlet": {
        "hideTitle": false,
        "intelligentCaptions": { "captions": [], "hasLoaded": false, "hiddenCaptions": [], "isExpanded": false },
        "isConfigVisible": true,
        "mfeJson": {
          "edges": [
            { "id": "xy-edge__node-aright-source-node-bleft-target", "type": "journey", "source": "node-a", "targe
t": "node-b", "sourceHandle": "right-source", "targetHandle": "left-target", "data": {} },
            { "id": "xy-edge__node-bright-source-node-cleft-target", "type": "journey", "source": "node-b", "targe
t": "node-c", "sourceHandle": "right-source", "targetHandle": "left-target", "data": {} }
          ],
          "nodes": [
            {
              "id": "node-a", "type": "component",
              "data": { "type": "Dimension item", "iconType": "dimensionItem", "components": [{ "id": "variables
/daterangeday::1260201", "itemType": "dimensionItem", "name": "Mar 1, 2026", "type": "DimensionItem" }] },
              "position": { "x": 16, "y": 208 }
            },
            {
              "id": "node-b", "type": "component",
              "data": { "type": "Dimension item", "iconType": "dimensionItem", "components": [{ "id": "variables
/daterangeday::1260202", "itemType": "dimensionItem", "name": "Mar 2, 2026", "type": "DimensionItem" }] },
              "position": { "x": 272, "y": 208 }
            },
            {
              "id": "node-c", "type": "component",
              "data": { "type": "Dimension item", "iconType": "dimensionItem", "components": [{ "id": "variables
/daterangeday::1260203", "itemType": "dimensionItem", "name": "Mar 3, 2026", "type": "DimensionItem" }] },
              "position": { "x": 528, "y": 208 }
            }
          ],
          "filters": { "selectedNodeTypes": ["Dimension item"], "arrowSetting": "labels", "percentageSetting": "pe
rcentoftotal", "showFallout": false },
          "container": "visitors",
          "primaryMetric": { "id": "metrics/visitors", "type": "Metric" },
          "secondaryMetric": { "id": "metrics/occurrences", "type": "Metric" },
          "loadFromAjoJourney": false,
          "isSequential": true,
          "stateViewport": { "x": -63, "y": -197, "zoom": 1.23 },
          "version": 21
        },
        "showAnnotations": true,
        "showControls": true,
        "type": "JourneyCanvasReportlet"
      },
      "swatchColor": "#00C0C7",
      "type": "genericSubPanel",
      "visible": true,
      "visualizationIndex": 1
    }
  ]
}
```


Key notes: Journey Canvas uses `linkedSourceId: ""` (self-contained). The `mfeJson` uses a React Flow graph — metrics use simplified `{ "id": ..., "type": "Metric" }` format (no `__entity__`). Node components use `itemType` instead of `__entity__`. Space nodes ~256px apart horizontally. Panel `autoHeight: 991`, subpanel `autoHeight: 817`.

14. Key Metric Summary

See the dedicated "Key Metric Summary Visualization" section above for full details. A KMS showing Events with this month vs previous month comparison:


```
{
  "subPanels": [
    {
      "id": "SP-KMS",
      "linkedSourceId": "SP-TABLE",
      "position": { "autoHeight": 475, "autoSize": true, "width": 100, "x": 0, "y": 0 },
      "reportlet": {
        "abbreviatePrecision": 1,
        "abbreviateValue": false,
        "anchorToLeft": false,
        "comparisonDateRange": {
```

```
          "__entity__": true, "type": "DateRange",
          "__metaData__": { "name": "Previous month", "definition": "tm-1m/tm" }
        },
        "config": {
          "comparisonDateRange": {
            "__entity__": true, "type": "DateRange",
            "__metaData__": { "name": "Previous month", "definition": "tm-1m/tm" }
          },
          "keyMetric": {
            "id": "metrics/occurrences", "__entity__": true, "type": "Metric",
            "__metaData__": { "name": "Events" }
          },
          "primaryDateRange": {
            "__entity__": true, "type": "DateRange",
            "__metaData__": { "name": "This month", "definition": "tm/tm+1m" }
          },
          "type": "reportletConfig"
        },
        "disabledLegends": [],
        "displayType": "change",
        "hasLimit": false,
        "intelligentCaptions": { "captions": [], "hasLoaded": false, "hiddenCaptions": [], "isExpanded": false },
        "isConfigVisible": true,
        "keyMetric": {
          "id": "metrics/occurrences", "__entity__": true, "type": "Metric",
          "__metaData__": { "name": "Events" }
        },
        "legendVisible": true,
        "lockType": "positions",
        "numItems": 14,
        "primaryDateRange": {
          "__entity__": true, "type": "DateRange",
          "__metaData__": { "name": "This month", "definition": "tm/tm+1m" }
        },
        "showAnnotations": true,
        "showComparison": true,
        "showControls": true,
        "showMax": false,
        "showMaxMin": false,
        "showMin": false,
        "showRawDifference": false,
        "showTrendlines": true,
        "showVizSelectorOnSubPanel": false,
        "showXAxis": true,
        "summaryDisplayType": "percent",
        "type": "KeyMetricSummaryReportlet",
        "usePercentage": false,
        "useRowBasedPercentages": false
      },
      "swatchColor": "#00C0C7",
      "type": "genericSubPanel",
      "visible": true,
      "visualizationIndex": 1
    },
    {
      "id": "SP-TABLE",
      "linkedSourceId": "",
      "position": { "autoHeight": 377, "autoSize": true, "width": 100, "x": 0, "y": 475 },
      "reportlet": {
        "type": "FreeformReportlet",
        "columnTree": {
          "id": "CT-ROOT", "name": "", "_computedValues": [], "visible": true,
          "nodes": [
            {
              "id": "COL-001", "_computedValues": [], "visible": true,
              "component": { "id": "metrics/occurrences", "__entity__": true, "type": "Metric", "__metaData__": {
"name": "Events" } },
              "name": "Events",
              "nodes": [
                {
                  "id": "COL-001-DR", "_computedValues": [], "visible": true,
                  "component": { "__entity__": true, "type": "DateRange", "__metaData__": { "name": "This month",
"definition": "tm/tm+1m" } },
                  "name": "This month", "nodes": []
                }
              ]
            },
            {
              "id": "COL-002", "_computedValues": [], "visible": true,
              "component": { "id": "metrics/occurrences", "__entity__": true, "type": "Metric", "__metaData__": {
"name": "Events" } },
              "name": "Events",
              "nodes": [
                {
```

```
                  "id": "COL-002-DR", "_computedValues": [], "visible": true,
                  "component": { "__entity__": true, "type": "DateRange", "__metaData__": { "name": "Previous
month", "definition": "tm-1m/tm" } },
                  "name": "Previous month", "nodes": []
                }
              ]
            }
          ]
        },
        "freeformTable": {
          "alignDatesForTimeDimension": true,
          "breakdowns": [],
          "columnWidths": [100, 100],
          "dimensionSettings": [
            { "dimension": { "id": "variables/daterangeday", "__entity__": true, "type": "Dimension", "__metaData_
_": { "name": "Day" } }, "id": "DIM-001" }
          ],
          "pagination": { "currentPage": 0, "viewBy": 400 },
          "sort": { "asc": true, "columnId": "", "labelColumn": true },
          "settings": { "breakdownByPosition": false, "rowBasedPercentages": false, "totalsType": "allVisits" },
          "staticRows": [],
          "statistics": { "functions": [], "ignoreZeros": true }
        }
      },
      "type": "genericSubPanel",
      "visible": false,
      "visualizationIndex": 1
    }
  ]
}
```


Key notes: `keyMetric`, `primaryDateRange`, and `comparisonDateRange` must be on both the reportlet AND inside `config`. The linked table needs **two metric columns**, each with a nested DateRange child matching the primary and comparison periods — without this, CJA shows "out of sync". Table is `visible: false`. Panel `autoHeight: 651`, KMS subpanel `autoHeight: 475`.

15. Map

See the dedicated "Map Visualization" section above for full details. A bubble map showing Events:


```
{
  "subPanels": [
    {
      "id": "SP-MAP",
      "linkedSourceId": "",
      "position": { "autoHeight": 477, "autoSize": true, "width": 50, "x": 0, "y": 0 },
      "reportlet": {
        "bearing": 0,
        "center": [-45.34, 29.3],
        "clusterRadius": 50,
        "colorTheme": "reds1",
        "customMaxValue": 0,
        "hideTitle": false,
        "intelligentCaptions": { "captions": [], "hasLoaded": false, "hiddenCaptions": [], "isExpanded": false },
        "isConfigVisible": true,
        "isLatLng": true,
        "mapStyle": "light",
        "mapType": "bubbles",
        "metricComponent": {
          "id": "metrics/occurrences", "__entity__": true, "type": "Metric",
          "__metaData__": { "name": "Events" }
        },
        "pitch": 0,
        "showAnnotations": true,
        "showControls": true,
        "showVizSelectorOnSubPanel": false,
        "type": "MapReportlet",
        "useCustomMaxValue": false,
        "useRowBasedPercentages": false,
        "zoom": 0.43
      },
      "swatchColor": "#2780EB",
      "type": "genericSubPanel",
      "visible": true,
      "visualizationIndex": 1
    }
  ]
}
```

Key notes: Map uses `linkedSourceId: ""` (self-contained). `metricComponent` is the metric entity. `isLatLng: true` is recommended. `center` is `[latitude, longitude]`. `zoom: 0.43` for world view. `width: 50` for half-width layout. Panel `autoHeight: 651`, subpanel `autoHeight: 477`.

16. Next or Previous Item Panel

See the "Panel-Level Configs" section above for full details. This is a **panel-level config** — no subPanels needed:

```
{
  "panels": [
    {
      "collapsed": false,
      "config": {
        "container": "visitors",
        "dimension": {
          "id": "variables/daterangeday", "__entity__": true, "type": "Dimension",
          "__metaData__": { "name": "Day" }
        },
        "dimensionItem": {
          "id": "variables/daterangeday::1260201", "__entity__": true, "type": "DimensionItem",
          "__metaData__": { "name": "Mar 1, 2026" }
        },
        "direction": "next",
        "registeredConfigs": {},
        "type": "NextOrPreviousItemPanelConfig"
      },
      "dateRange": { "id": "thisMonth", "__entity__": true, "type": "DateRange", "__metaData__": { "name": "This
month" } },
      "datesAreRelativeToPanel": false,
      "description": "",
      "id": "PANEL-001",
      "name": "Next or previous item",
      "position": { "autoHeight": 536, "autoSize": true, "width": 100, "x": 0, "y": 0 },
      "reportSuite": {
        "id": "dv_YOUR_DATA_VIEW_ID", "__entity__": true, "type": "ReportSuite",
        "__metaData__": { "name": "Analytics Streaming data", "rsid": "dv_YOUR_DATA_VIEW_ID" }
      },
      "segmentGroups": [],
      "subPanels": [],
      "type": "panel"
    }
  ]
}
```

Key notes: The `config` is on the **panel** itself (not a reportlet). `subPanels` must be `[]` — the panel auto-generates its visualizations. `direction: "next"` shows what came after; `"previous"` shows what came before. `dimensionItem` needs a real DimensionItem ID. Panel `autoHeight: 536`.

17. Quick Insights Panel

See the "Panel-Level Configs > quickInsightsPanelConfig" section above for full details. This is a **panel-level config** with pre-built subPanels:

```
{
  "panels": [
    {
      "collapsed": false,
      "config": {
        "dateRange": { "id": "thisMonth", "__entity__": true, "type": "DateRange", "__metaData__": { "name": "This
month" } },
        "dimensions": {
          "first": { "show": true }, "hideFirstItem": false,
          "second": { "show": false }, "third": { "show": false }
        },
        "metrics": {
          "first": { "show": true }, "hideFirstItem": false,
          "second": { "show": false }, "third": { "show": false }
        },
        "registeredConfigs": {},
        "segments": {
          "first": { "logicalOperator": "and", "show": true }, "hideFirstItem": false,
          "second": { "logicalOperator": "and", "show": false },
          "third": { "logicalOperator": "and", "show": false }
        },
        "type": "quickInsightsPanelConfig"
      },
      "dateRange": { "id": "thisMonth", "__entity__": true, "type": "DateRange", "__metaData__": { "name": "This
month" } },
      "datesAreRelativeToPanel": false,
      "description": "",
      "id": "PANEL-001",
      "name": "Quick insights",
      "position": { "autoHeight": 837, "autoSize": true, "width": 100, "x": 0, "y": 0 },
      "reportSuite": {
        "id": "dv_YOUR_DATA_VIEW_ID", "__entity__": true, "type": "ReportSuite",
        "__metaData__": { "name": "Analytics Streaming data", "rsid": "dv_YOUR_DATA_VIEW_ID" }
      },
      "segmentGroups": [],
      "subPanels": [
        {
          "id": "quickinsights-freeform-subpanel", "isQuickInsightsSubPanel": true,
          "linkedSourceId": "",
          "position": { "autoHeight": 419, "autoSize": true, "width": 50, "x": 0, "y": 0 },
          "reportlet": { "type": "FreeformReportlet", "columnTree": { "id": "CT-ROOT", "name": "", "nodes": [], "v
isible": true }, "freeformTable": { "columnWidths": [50, 50], "breakdowns": [], "dimensionSettings": [] } },
          "type": "genericSubPanel", "visible": true
        },
        {
          "id": "quickinsights-viz-subpanel", "isQuickInsightsSubPanel": true,
          "linkedSourceId": "quickinsights-freeform-subpanel",
          "position": { "autoHeight": 419, "autoSize": false, "fixedHeight": 419, "width": 50, "x": 50, "y": 0 },
          "reportlet": { "type": "BarReportlet", "hasLimit": false, "numItems": 14 },
          "type": "genericSubPanel", "visible": true
        }
      ],
      "type": "panel"
    }
  ]
}
```


Key notes: `config.type` is camelCase `"quickInsightsPanelConfig"` (NOT PascalCase). Both subpanels need `isQuickInsightsSubPanel: true` with exact IDs `"quickinsights-freeform-subpanel"` and `"quickinsights-viz-subpanel"`. The freeform table is empty — CJA populates it on interaction. Panel `autoHeight: 837`.

18. Venn Diagram

See the "Venn Visualization" section above for full details. Venn holds its own `metrics` and `segments` and links to a hidden empty table:

```
{
  "subPanels": [
    {
      "id": "SP-VENN", "isQuickInsightsSubPanel": false,
      "linkedSourceId": "SP-TABLE",
      "position": { "autoHeight": 377, "autoSize": true, "width": 100, "x": 0, "y": 0 },
      "reportlet": {
        "type": "VennReportlet",
        "contextForSegmentConversion": "visits",
        "lockType": "positions",
        "metrics": [
          { "id": "metrics/occurrences", "__entity__": true, "type": "Metric", "__metaData__": { "name": "Events"
} }
        ],
        "segments": [
          { "id": "All_Visits", "__entity__": true, "type": "Segment", "__metaData__": { "name": "All Data" } }
        ],
        "normalized": false,
        "showVizSelectorOnSubPanel": false
      },
      "type": "genericSubPanel", "visible": true
    },
    {
      "id": "SP-TABLE", "isQuickInsightsSubPanel": false,
      "linkedSourceId": "", "name": "Venn Data",
      "position": { "autoSize": true, "width": 100, "x": 0, "y": 0 },
      "reportlet": {
        "type": "FreeformReportlet",
        "columnTree": { "id": "CT-ROOT", "name": "", "nodes": [], "visible": true },
        "freeformTable": { "columnWidths": [100], "breakdowns": [], "dimensionSettings": [] },
        "name": "Venn Data"
      },
      "type": "genericSubPanel", "visible": false
    }
  ]
}
```

Key notes: Each segment in the `segments` array becomes a Venn circle. `metrics` typically has one metric. The linked FreeformReportlet is empty and hidden (`visible: false`). Panel `autoHeight: 551`.

19. Dashboard Grid Layout (Multi-Row, Multi-Column)

A single-panel dashboard with summary tiles across the top, a tall bar chart on the left spanning two rows, and a horizontal bar chart above a freeform table on the right. Hidden data-source subpanels back each visualization.

```
Summary     Text    Summary

             Horizontal Bar
 Bar Chart
 (tall)      Freeform Table
```

```
{
  "subPanels": [
    {
      "id": "SP-SUM1", "linkedSourceId": "SP-D1",
      "position": { "autoSize": true, "width": 33.33, "x": 0, "y": 0 },
      "reportlet": { "type": "SummaryNumberReportlet", "lockedSelection": { "breakdowns": [], "selectedDataPoints"
: [{ "columnId": "C1-1", "rowPosition": -1 }], "type": "FreeformBreakdownSelection" }, "lockType": "positions" },
      "type": "genericSubPanel", "visible": true, "visualizationIndex": 1
    },
    {
      "id": "SP-TXT", "linkedSourceId": "",
      "position": { "autoSize": true, "width": 33.34, "x": 33.33, "y": 0 },
      "reportlet": { "type": "TextReportlet", "textContent": "{\"ops\":[{\"attributes\":{\"bold\":true},\"
insert\":\"Dashboard Title\"},{\"insert\":\"\\nDescription text here.\\n\"}]}", "hideTitle": true },
      "type": "genericSubPanel", "visible": true, "visualizationIndex": 2
    },
    {
      "id": "SP-SUM3", "linkedSourceId": "SP-D3",
      "position": { "autoSize": true, "width": 33.33, "x": 66.67, "y": 0 },
      "reportlet": { "type": "SummaryNumberReportlet", "lockedSelection": { "breakdowns": [], "selectedDataPoints"
: [{ "columnId": "C3-1", "rowPosition": -1 }], "type": "FreeformBreakdownSelection" }, "lockType": "positions" },
      "type": "genericSubPanel", "visible": true, "visualizationIndex": 3
    },
    {
      "id": "SP-BAR", "linkedSourceId": "SP-D-BAR",
      "position": { "autoSize": false, "fixedHeight": 650, "width": 50, "x": 0, "y": 0 },
      "reportlet": { "type": "BarReportlet", "numItems": 14, "lockType": "positions", "legendVisible": true, "norm
alized": false },
      "type": "genericSubPanel", "visible": true, "visualizationIndex": 4
    },
    {
      "id": "SP-HBAR", "linkedSourceId": "SP-D-HBAR",
      "position": { "autoSize": true, "width": 50, "x": 50, "y": 0 },
      "reportlet": { "type": "HorizontalBarReportlet", "numItems": 10, "lockType": "positions", "legendVisible": t
rue, "normalized": false },
      "type": "genericSubPanel", "visible": true, "visualizationIndex": 5
    },
    {
      "id": "SP-TABLE", "linkedSourceId": "SP-D-TABLE",
      "position": { "autoSize": true, "width": 50, "x": 50, "y": 0 },
      "reportlet": { "type": "FreeformReportlet", "columnTree": { "..." : "visible table" }, "freeformTable": { ".
.." : "with dimension & metrics" } },
      "type": "genericSubPanel", "visible": true, "visualizationIndex": 6
    },
    { "id": "SP-D1", "linkedSourceId": "", "visible": false, "visualizationIndex": 7, "reportlet": { "type": "Free
formReportlet", "..." : "data for SP-SUM1" }, "type": "genericSubPanel" },
    { "id": "SP-D3", "linkedSourceId": "", "visible": false, "visualizationIndex": 8, "reportlet": { "type": "Free
formReportlet", "..." : "data for SP-SUM3" }, "type": "genericSubPanel" },
    { "id": "SP-D-BAR", "linkedSourceId": "", "visible": false, "visualizationIndex": 9, "reportlet": { "type": "F
reeformReportlet", "..." : "data for SP-BAR" }, "type": "genericSubPanel" },
    { "id": "SP-D-HBAR", "linkedSourceId": "", "visible": false, "visualizationIndex": 10, "reportlet": { "type":
"FreeformReportlet", "..." : "data for SP-HBAR" }, "type": "genericSubPanel" },
    { "id": "SP-D-TABLE", "linkedSourceId": "", "visible": false, "visualizationIndex": 11, "reportlet": { "type":
"FreeformReportlet", "..." : "data for SP-TABLE" }, "type": "genericSubPanel" }
  ]
}
```

Key notes:

- All subpanels use `y: 0` — CJA auto-computes vertical positions
- The bar chart uses `autoSize: false` + `fixedHeight: 650` to span the height of both right-column rows
- Right-column subpanels (`x: 50`) flow vertically: horizontal bar stacks above the freeform table
- Hidden data-source subpanels (`visible: false`) back each visualization via `linkedSourceId`
- The visible freeform table (`SP-TABLE`) is both a visualization and its own data source — it has `linkedSourceId` pointing to a separate hidden table, OR it can use `linkedSourceId: ""` if it is self-contained

---

Best Practices

1. Always Include Required Fields

- Every Entity must have `__entity__: true`, `id`, and `type`
- Every project definition must have `version` and `workspaces`
- Every panel must have `type: "panel"` and every subpanel must have `type: "genericSubPanel"`

2. Use UUIDs for All IDs

- Generate unique UUIDs for workspace, panel, subpanel, and column node IDs
- Column node IDs are referenced by `sort.columnId` and breakdown `parentItemId`

3. Match linkedSourceId Correctly

- The chart subpanel's `linkedSourceId` must exactly match the data subpanel's `id`
- A data subpanel (FreeformReportlet) always has `linkedSourceId: ""`

4. Use breakdownByPosition Intentionally

- Default to `true` for dynamic dashboards where rows may change
- Use `false` when breakdowns must always target a specific, known item

5. Validate Before Saving

- Always validate the definition JSON against the schema before calling `upsertProject`
- See the Validation section below

6. Keep columnWidths Aligned

- The `columnWidths` array length should match the number of visible columns (dimension label + metrics)

Validation

Recommended: npx ajv-cli (supports $ref)

The project schema uses 350+ internal `$ref` references with `additionalProperties: false`. Most online validators and simple curl APIs **cannot** res olve `$ref` and will produce false errors. Use `ajv-cli` which properly handles Draft 4 + `$ref`:

```
# Download the v96 schema (requires --compressed for CDN)
curl -s --compressed -o /tmp/projectSchema_v96.json \
  "https://awaascicdprodva7.blob.core.windows.net/deploy/schemas/project/projectSchema_v96.json"

# Validate your project JSON
npx ajv-cli@3.3.0 validate -s /tmp/projectSchema_v96.json -d your_project.json
```

Output: `your_project.json valid` or detailed error messages with JSON path.

Online Validator

Paste the schema and your JSON at: jsonschemavalidator.net

> ***Note**: Online validators may not resolve `$ref` correctly for this schema.*

curl (Assertible API — limited)

The Assertible API is a simple curl-based validator but **does not support `$ref`**. It will produce false "additional properties" errors for this schema. Only use for schemas without `$ref`:

```
curl -s "https://assertible.com/json" \
  -H "Content-Type: application/json" \
  -d '{"schema": {"type": "object"}, "json": {}}'
```

Python (local)

```
pip install jsonschema
```

```python
import json
from jsonschema import validate, ValidationError

with open("projectSchema_v96.json") as f:
    schema = json.load(f)

with open("your_project.json") as f:
    instance = json.load(f)

try:
    validate(instance=instance, schema=schema)
    print("Valid!")
except ValidationError as e:
    print(f"Invalid: {e.message}")
```

Common Errors and Troubleshooting

1. Missing `__entity__: true`

Every component reference must include `"__entity__": true`. Without it, the entity won't resolve.

2. Wrong Entity Type

- Metrics use `"type": "Metric"`, not `"Metrics"` or `"metric"`
- Dimensions use `"type": "Dimension"`, not `"Dimensions"`
- The `type` values are case-sensitive

3. linkedSourceId Mismatch

If a chart shows no data, verify that its `linkedSourceId` exactly matches the data subpanel's `id`.

4. Invalid Sort columnId

The `sort.columnId` must reference an existing column node `id` in the `columnTree`. An empty string with `labelColumn: true` sorts alphabetically by dimension labels.

5. Schema Validation Fails on additionalProperties

The schema uses `"additionalProperties": false` on every object. Any extra field not defined in the schema will cause validation failure.

6. Bare Definition Without projectBody Wrapper

The most common reason a project "doesn't work" is passing the bare definition JSON (`{ "version": "96", "workspaces": [...] }`) instead of wrapping it in a projectBody (`{ "definition": { ... }, "rsid": "dv_...", "type": "project" }`). The API requires the wrapper.

7. Quill Delta Escaping

SubPanel `description` and TextReportlet `textContent` values are JSON strings containing serialized JSON. The inner JSON must be escaped: `\"` for quotes, `\\n` for newlines.

8. lockedSelection: "None of the provided types can parse this json"

The `lockedSelection` object is an `anyOf` union between `FreeformBreakdownSelection` and `CohortSelection`. The parser requires a `"type"` di scriminator field to determine which type to use. If `"type": "FreeformBreakdownSelection"` is missing from any level of the object, this error occurs. Every `FreeformBreakdownSelection` object in the tree (top-level and nested breakdowns) must include it.

9. Multi-Dimension Table Fails or Shows Wrong Layout

When using multiple dimensions in `dimensionSettings`, the `columnWidths` array must have entries for **all** columns: one per dimension label column plus one per metric column. For example, 2 dimensions + 1 metric = 3 `columnWidths` entries. If the count is wrong, the table may fail to render or display with broken column sizing. Also ensure the table subpanel's `position.y` is offset by the height of any visualization above it (e.g., `y: 377` for a standard chart).

10. Flow: "Cannot read properties of undefined (reading 'numColumns')"

The Flow reportlet requires a `config` object with `"type": "reportletConfig"` and `numColumns`. Without the `config` object, or if it's missing `numColumns`, the flow will fail. Also, if `expandedNodes` at level 1 is empty, the flow renders only one node (the root) without expanding to additional columns. Level 1 needs a Dimension entity, and level 2 needs DimensionItem entities with real IDs. See the "Flow Visualization" section for the full `expandedNodes` pattern.

11. Summary Number Selecting Multiple Cells

If a summary number displays an unexpected aggregation or shows "multiple items selected", the `lockedSelection` likely has `selectedDataPoints` entries at **both** the top level and inside a `breakdowns` entry. When pointing to a breakdown row, the **top-level `selectedDataPoints` must be `[]`** (empty). Only the specific target breakdown entry should have a non-empty `selectedDataPoints`. See the "lockedSelection & FreeformBreakdownSelection" section for the correct pattern.

Template-Specific Features (autorun & autoCompareTimePeriod)

Some features are **template-only** — they work when a project is loaded as a template but are not standard project definition properties. Understanding these is important to avoid confusion about what's possible in regular project JSON.

autorun (Panel Config Property)

`autorun` is a property on **panel configs** (not reportlets) that causes the panel to automatically execute after configuration. Only **three** panel config types support it:

Panel Config Type	Description
`AnalyticsForTargetPanelConfig`	Analytics for Target (A4T) panels
`AudienceComparisonPanelConfig`	Segment Comparison panels
`ExperimentPanelConfig`	Experimentation panels

What does NOT support autorun:

- `NextOrPreviousItemPanelConfig` — no autorun property
- `AttributionPanelConfig` — no autorun property
- `HistogramReportlet` — this is a reportlet (visualization), not a panel config; autorun doesn't apply
- `Flow` / `Fallout` / `CohortReportlet` — these are reportlets, not panel configs

`autorun` is consumed in the panel config UI code (`PanelConfig.js`) to control whether to show the config panel or auto-trigger the build. It's not currently set in any of the public template JSON files, but the code supports it being passed via `data.autorun` for the three panel types above.

autoCompareTimePeriod (Column Node Property — Templates Only)

`autoCompareTimePeriod` is a property on **freeform table column nodes** (inside `columnTree.nodes`) that automatically adds a date comparison column with percent change when the template loads.

```
{
  "id": "COL-001",
  "component": { "id": "metrics/occurrences", "__entity__": true, "type": "Metric" },
  "name": "Events",
  "nodes": [],
  "autoCompareTimePeriod": {
    "dateRange": "td-29d-30d/td+1d-30d"
  }
}
```

Field	Type	Description
`autoCompareTimePeriod.dateRange`	string	Date range definition string for the comparison period (e.g.,`"td-29d-30d/td+1d-30d"` for prior 30-day period)

How it works:

1. `TemplateBase.onLoaded()` iterates all subpanels and calls `reportlet.autorunTemplateFunctions()`
2. Only `FreeformReportlet` implements `autorunTemplateFunctions` — all other reportlet types skip this
3. `FreeformReportlet.autorunCompareTimePeriods()` iterates leaf column nodes looking for `autoCompareTimePeriod`
4. For each match, it auto-adds a comparison column with percent change using the specified date range

Limitations:

- Only works in JS-based templates (e.g., `content-analytics.js`) — not currently used in any JSON public templates

- Only `FreeformReportlet` implements `autorunTemplateFunctions`; Histogram, Map, Text, etc. do not
- The comparison column is added dynamically at template load time, not persisted in the JSON definition

autorunTemplateFunctions Flow

```
TemplateBase.onLoaded()
   iterates all subPanels
     calls subPanel.reportlet?.autorunTemplateFunctions?.()
       FreeformReportlet.autorunTemplateFunctions()
         autorunCompareTimePeriods()
           checks each leaf node for autoCompareTimePeriod
           adds comparison range with percent change
```

Only `FreeformReportlet` implements this method. All other reportlet types return `undefined` via optional chaining and are skipped.

API Reference

For programmatic project management, use these MCP tools:

- **`upsertProject(projectBody, expansions, projectId?)`** — Create a new project (omit `projectId`) or update an existing one. The `projectBody` must be a wrapper object containing `definition` (the project definition JSON described in this guide), `rsid` (data view ID), and `type: "project"`. Do **not** pass the bare definition — it must be nested inside the `definition` field of the wrapper. See the "projectBody Wrapper" section above.
- **`findProjects(page, limit, expansions)`** — List projects available to the user. Use `expansions: "definition"` to include the full project JSON.
- **`describeProject(id, expansions)`** — Get full details for a specific project. Use `expansions: "definition"` to retrieve the project definition JSON.

Useful Expansions

Expansion	Description
`definition`	Full project definition JSON
`ownerFullName`	Project owner's name
`modified`	Last modified timestamp
`shares`	Share settings
`tags`	Project tags

Note: This guide covers the standard project definition patterns based on schema version 96. The schema source of truth is at `https://awaascicdprodva7.blob.core.windows.net/deploy/schemas/project/projectSchema_v96.json`. Advanced configurations (Guided Analysis panels) have complex type-specific settings not fully detailed here — inspect existing projects via `describeProject` to understand their structure.

Future Tasks

The following items are known gaps or improvements to explore:

1. Histogram — Autorun Support (Resolved: Not Supported)

Histograms are **reportlets** (visualizations), not panel configs. `autorun` is a panel config-level property and does not apply to reportlets. Histogram renders on project load with whatever configuration is provided — there is no autorun mechanism. See the "Template-Specific Features" section for details on which panel types support autorun.

2. Flow — Autorun Support (Resolved: Not Supported)

Flow is a **reportlet**, not a panel config, so `autorun` does not apply. Flow visualizations require pre-populated `expandedNodes` with specific `DimensionItem` IDs for multi-column expansion. There is no autorun mode. To make Flow definitions portable, create with empty `expandedNodes` at level 2 (renders 2 columns) and let CJA populate the IDs on interaction, then export.

3. Attribution — Full Documentation

The `AttributionPanelConfig` is a panel-level config (like NextOrPreviousItem) and does **not** support autorun. The `AttributionTemplate` entity type and attribution-related settings still need documentation:

- How `AttributionPanelConfig` works (panel-level config with `dimension`, `successMetric`, `lookbackExpiration`, `attributions`, `context`)
- How to apply attribution models to metrics in FreeformReportlet column nodes
- How `AttributionTemplate` entities work (e.g., `"allocation-instance"`)
- Whether attribution panels auto-generate subPanels like NextOrPreviousItem

4. Experimentation, A4T, and Segment Comparison Panels

These three panel types support `autorun` and are the only ones that do. Document:

- `ExperimentPanelConfig` — experimentation panel structure and autorun behavior
- `AnalyticsForTargetPanelConfig` — A4T panel structure and autorun behavior
- `AudienceComparisonPanelConfig` — segment comparison panel structure and autorun behavior